



UNIVERSITÀ DEGLI STUDI DI SALERNO



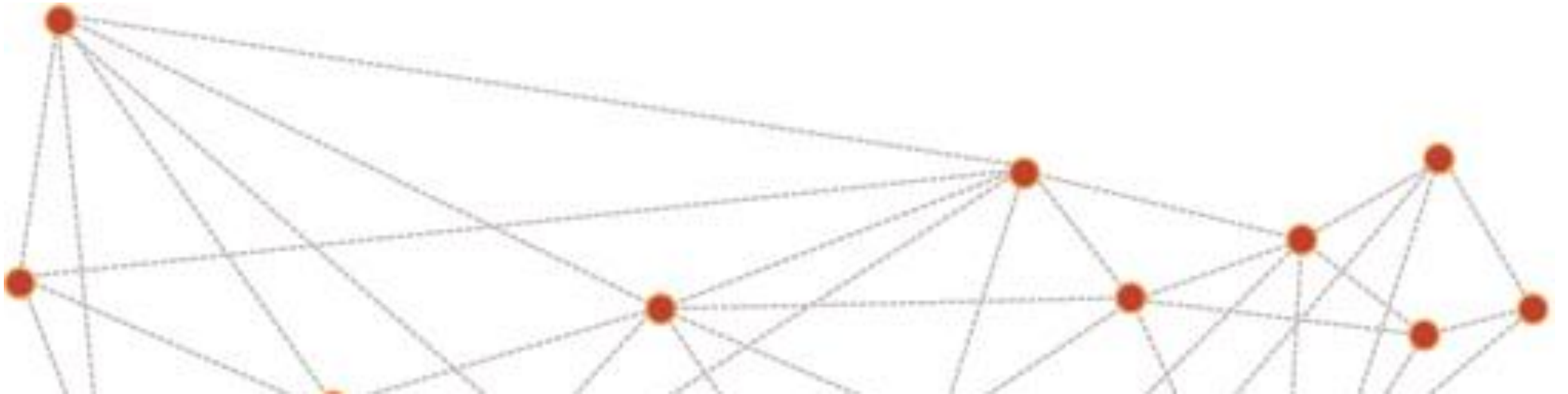
UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
DIPARTIMENTO DI ECCELLENZA



Prof. Carmen Bisogni

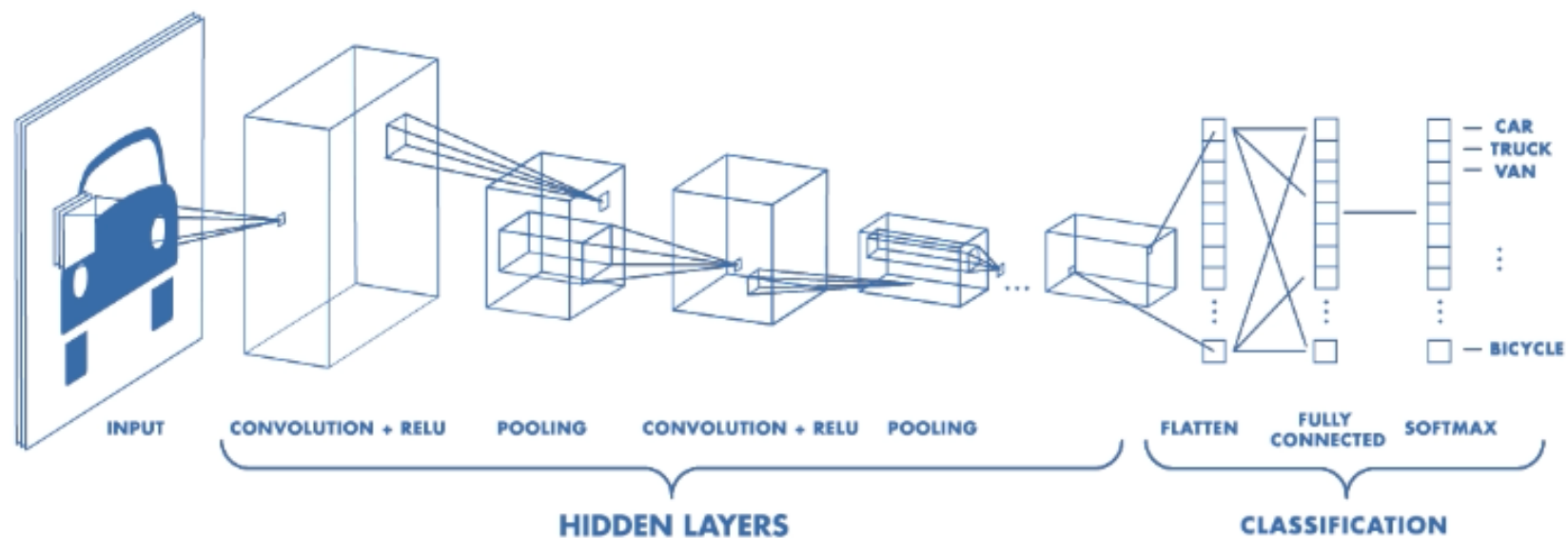
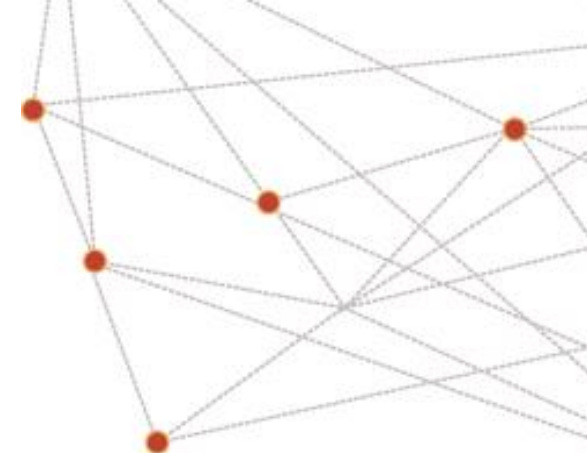
C.A.S.A. Course

CONTEXT AWARE SECURITY ANALYTICS IN COMPUTER VISION





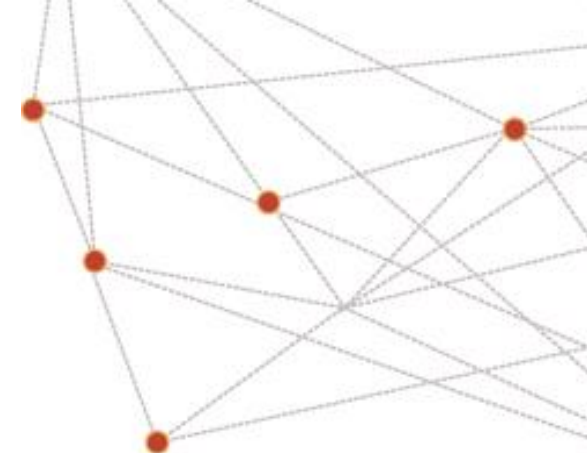
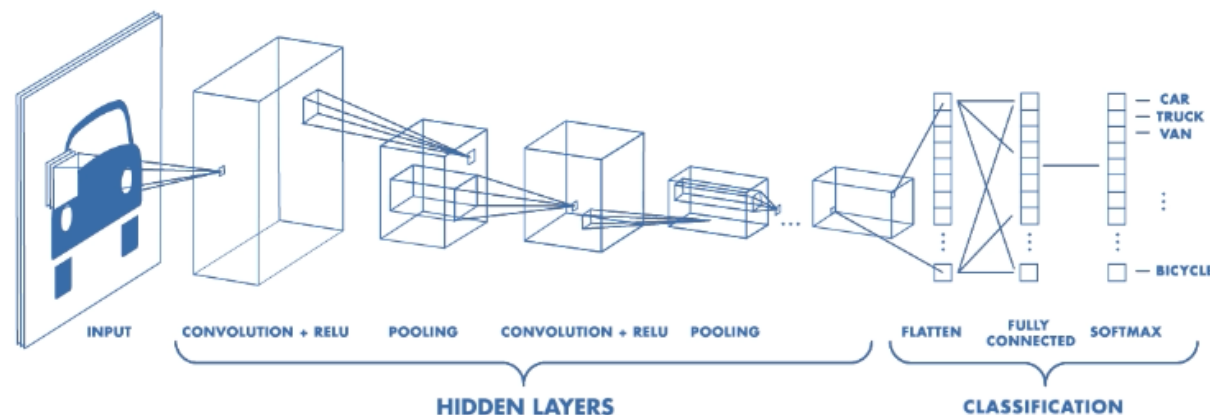
CNN structure





CNN structure in python

```
#defining model
model=Sequential()
#adding convolution layer
model.add(Conv2D(32,(3,3),activation='relu',
input_shape=(28,28,1)))
#adding pooling layer
model.add(MaxPool2D(2,2))
#adding fully connected layer
model.add(Flatten())
model.add(Dense(100,activation='relu'))
#adding output layer
model.add(Dense(10,activation='softmax'))
```

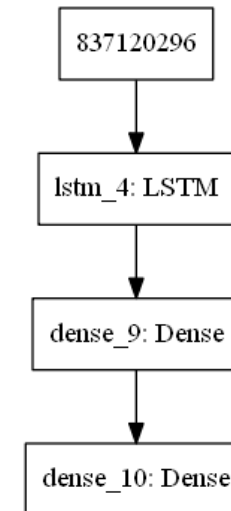


What means sequential?

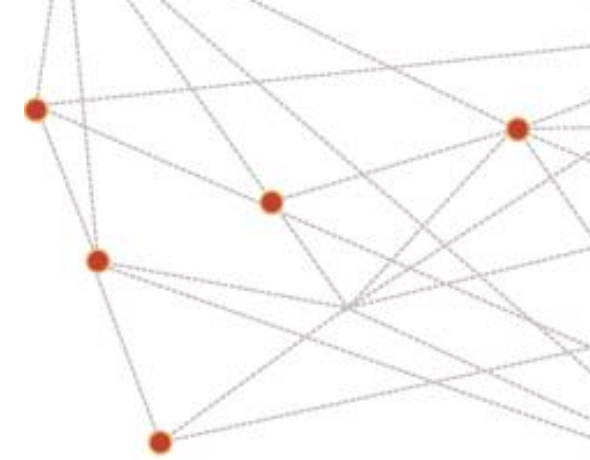
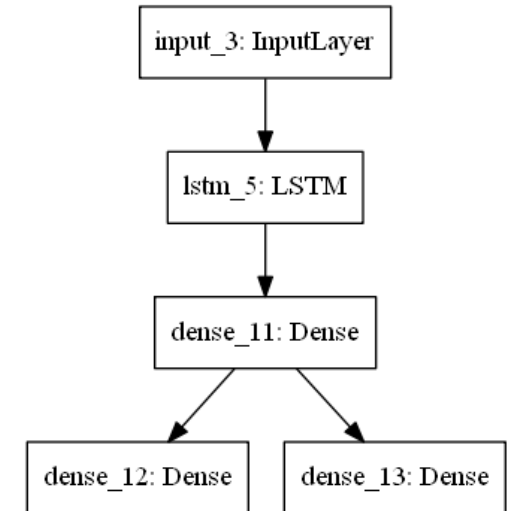
Keras models can be built using either the sequential or functional API.

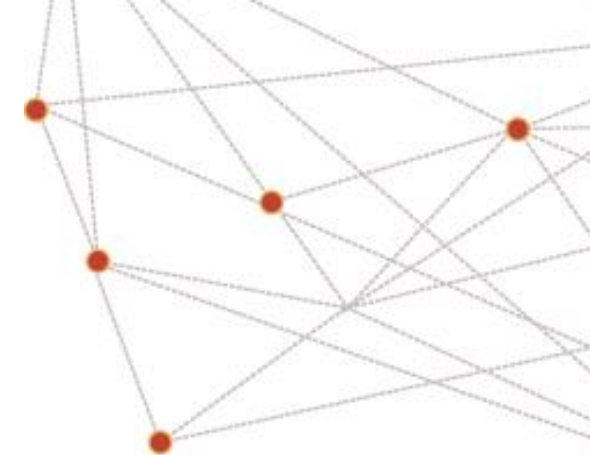
- The sequential API is simpler and allows you to create models layer-by-layer for most problems. However, it doesn't allow for shared layers or multiple inputs/outputs.
- The functional API is more flexible and allows you to connect layers to any other layer, making it possible to create complex networks like siamese and residual networks.

Sequential API



Functional API





How many parameters a Conv layer has?

`model.add(Conv2D(filters=16,`

Is an integer, the dimensionality of the output space (i.e. the number of output filters in the convolution).

`kernel_size=(3,3),`

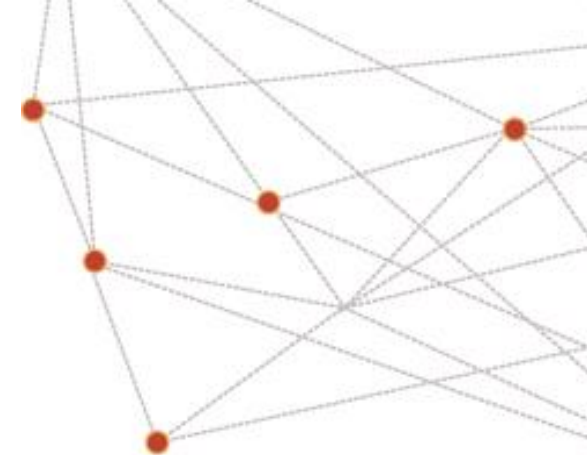
An integer or tuple/list of 2 integers, specifying the height and width of the 2D convolution window. Can be a single integer to specify the same value for all spatial dimensions.

`strides=(1,1),`

Stride is the number of pixels shifts over the input matrix. When the stride is 1 then we move the filters to 1 pixel at a time. When the stride is 2 then we move the filters to 2 pixels at a time and so on.

`padding='valid',`

Default='valid', "valid" means no padding. "same" results in padding with zeros evenly to the left/right or up/down of the input. When padding="same" and strides=1, the output has the same size as the input.



How many parameters a Conv layer has?

`data_format=None,`

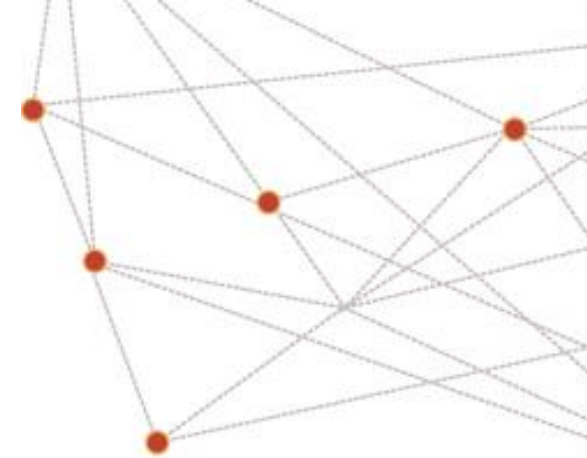
Default=None, A string, one of `channels_last` (default) or `channels_first`. The ordering of the dimensions in the inputs. `channels_last` corresponds to inputs with shape (batch_size, height, width, channels) while `channels_first` corresponds to inputs with shape (batch_size, channels, height, width).

`dilation_rate=(1, 1),`

Dilated Convolutions are a type of convolution that “inflate” the kernel by inserting holes between the kernel elements. An additional parameter (dilation rate) indicates how much the kernel is widened. There are usually spaces inserted between kernel elements.

`groups=1,`

Default=1, A positive integer specifying the number of groups in which the input is split along the channel axis. Each group is convolved separately with filters / groups filters. The output is the concatenation of all the groups results along the channel axis. Input channels and filters must both be divisible by groups.



How many parameters a Conv layer has?

`activation='relu',`

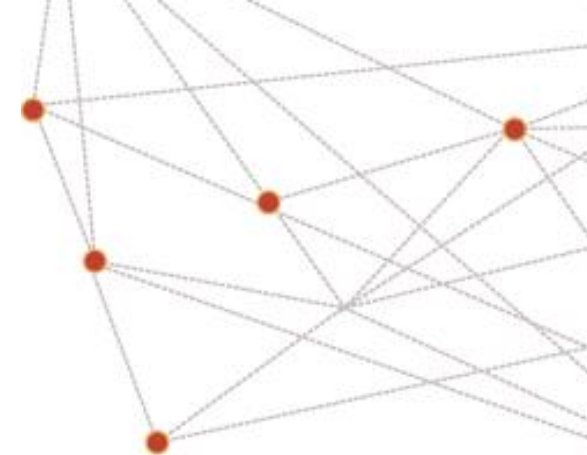
Default=None, Activation function to use. If you don't specify anything, no activation is applied. If you do not specify an activation for a layer you are effectively creating a linear transformation through that layer.

`use_bias=True,`

The bias is an additive parameter in the convolution. In a large model, removing the bias inputs makes very little difference because each node can make a bias node out of the average activation of all of its inputs, which by the law of large numbers will be roughly normal.

`kernel_initializer='glorot_uniform',`

Initializers define the way to set the initial random weights of Keras layers. Xavier/Glorot Initialization is used to maintain the same smooth distribution for both the forward pass as well the backpropagation.



How many parameters a Conv layer has?

`bias_initializer='zeros',`

It is possible and common to initialize the biases to be zero, since the asymmetry breaking is provided by the small random numbers in the weights. For ReLU non-linearities, some people like to use small constant value such as 0.01 for all biases because this ensures that all ReLU units fire in the beginning and therefore obtain and propagate some gradient.

`kernel_regularizer=None,
bias_regularizer=None,
activity_regularizer=None,`

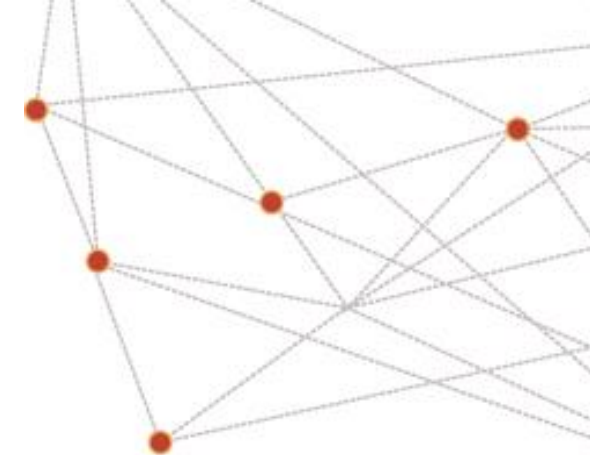
Regularizers allow you to apply penalties on layer parameters or layer activity during optimization. These penalties are summed into the loss function that the network optimizes. Regularization penalties are applied on a per-layer basis.

`kernel_constraint=None,
bias_constraint=None,`

Constraining the weight matrix directly is another kind of regularization. If you use a simple L2 regularization term you penalize high weights with your loss function. With this constraint, you regularize directly.



How many parameters a Max pooling layer has?



```
model.add(MaxPool2D  
(pool_size=(2,2),
```

```
strides=(2,2),
```

```
padding='valid',
```

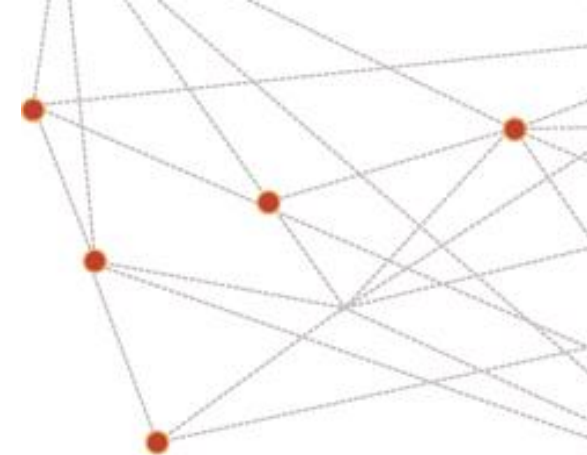
Default=(2,2), integer or tuple of 2 integers, window size over which to take the maximum. (2, 2) will take the max value over a 2x2 pooling window. If only one integer is specified, the same window length will be used for both dimensions.

Default=None, Integer, tuple of 2 integers, or None. Strides values. Specifies how far the pooling window moves for each pooling step. If None, it will default to pool_size. When the stride is 1 then we move the filters to 1 pixel at a time. When the stride is 2 then we move the filters to 2 pixels at a time and so on.

Default='valid', One of "valid" or "same" (case-insensitive). "valid" means no padding. "same" results in padding evenly to the left/right or up/down of the input such that output has the same height/width dimension as the input.



How many parameters a Max pooling layer has?

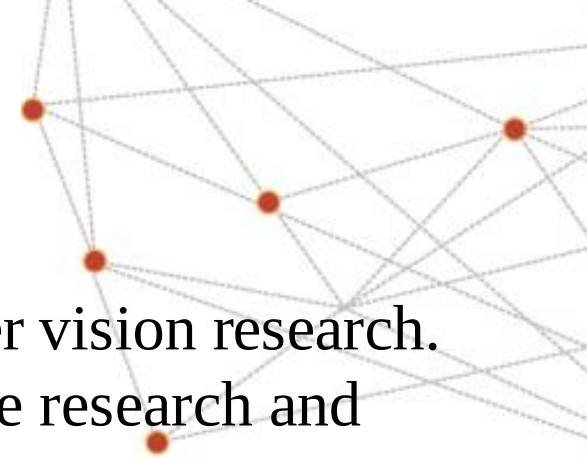


`data_format=None,`

Default=None, A string, one of `channels_last` (default) or `channels_first`. The ordering of the dimensions in the inputs. `channels_last` corresponds to inputs with shape (batch_size, height, width, channels) while `channels_first` corresponds to inputs with shape (batch_size, channels,height, width).

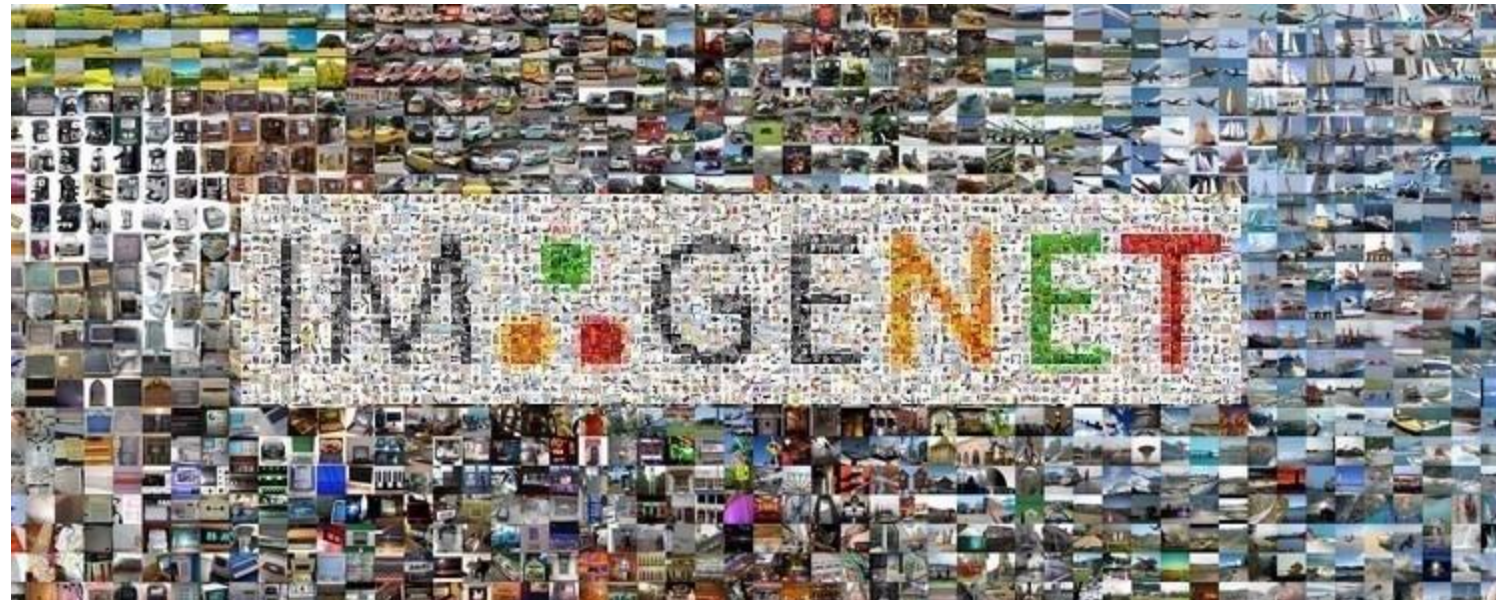


ImageNet Dataset



ImageNet is a large dataset of annotated photographs intended for computer vision research. The goal of developing the dataset was to provide a resource to promote the research and development of improved methods for computer vision.

There are a little more than 14 million images in the dataset, a little more than 21 thousand groups or classes and a little more than 1 million images that have bounding box annotations. The photographs were annotated by humans using crowdsourcing platforms such as Amazon's Mechanical Turk.



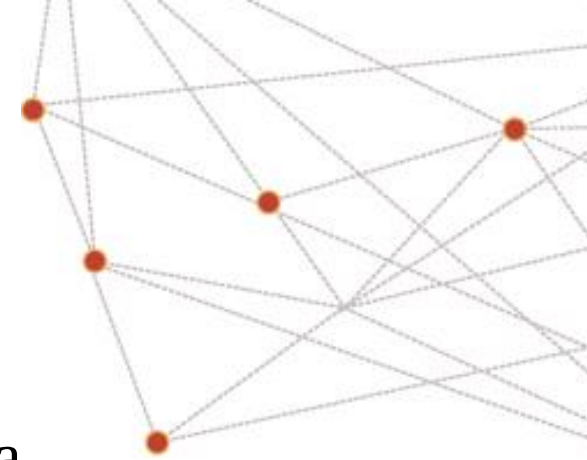


ImageNet Large Scale Visual Recognition Challenge

The ILSVRC is an annual computer vision competition developed upon a subset of ImageNet. As such, the tasks and even the challenge itself is often referred to as the ImageNet Competition.

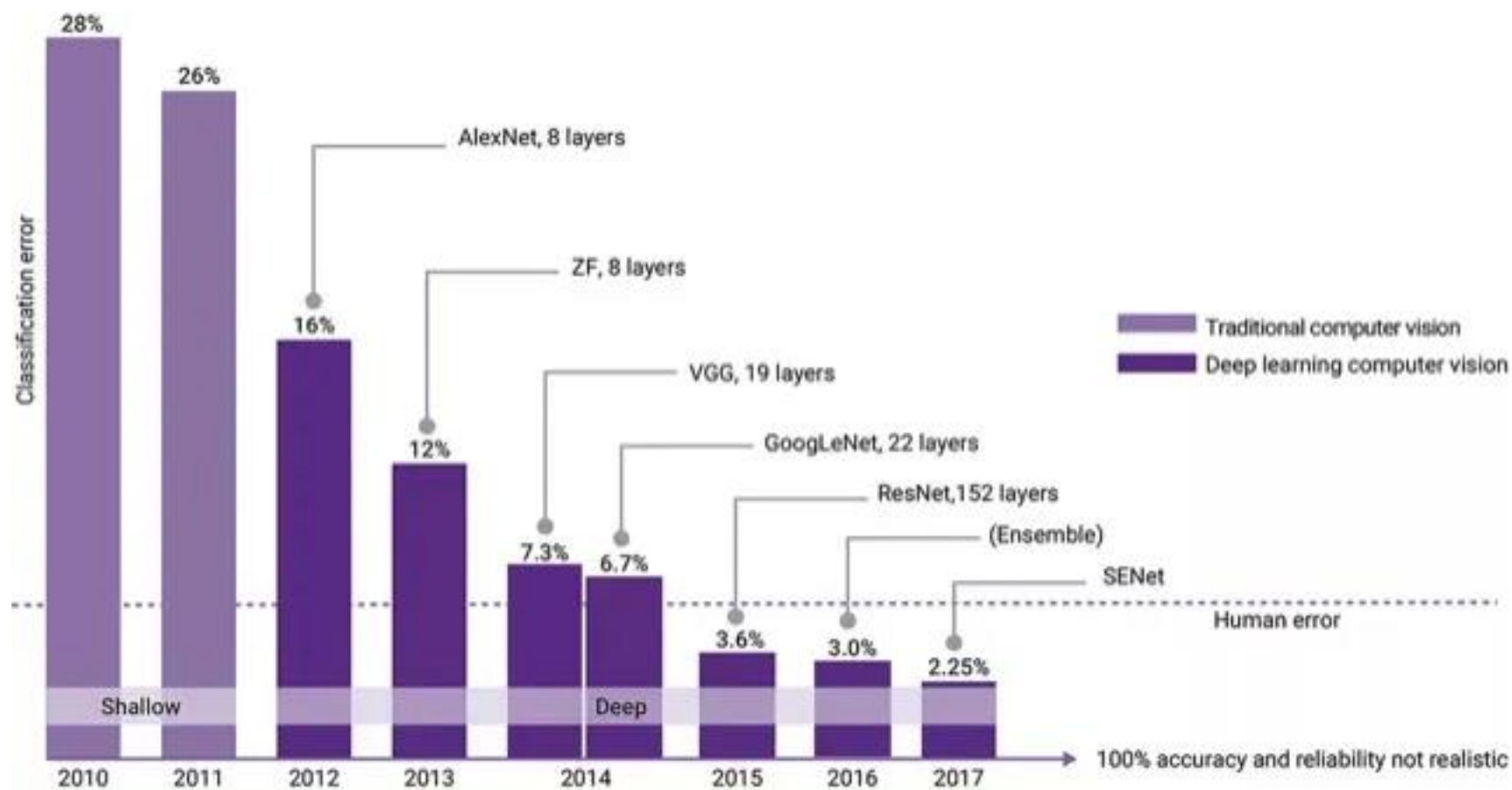
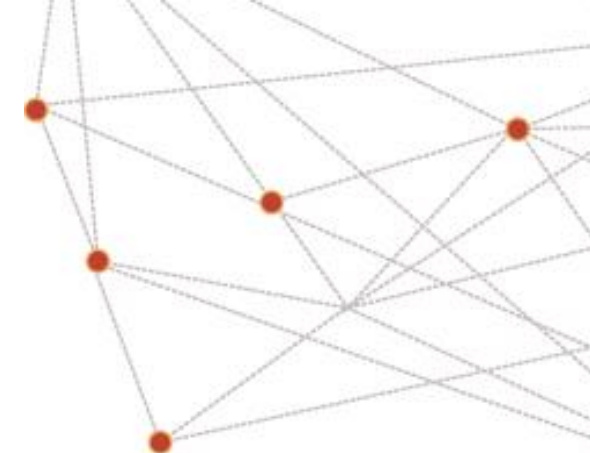
The general challenge tasks for most years are as follows:

- Image classification: Predict the classes of objects present in an image.
- Single-object localization: Image classification + draw a bounding box around one example of each object present.
- Object detection: Image classification + draw a bounding box around each object present.





Deep Learning Milestones From ILSVRC

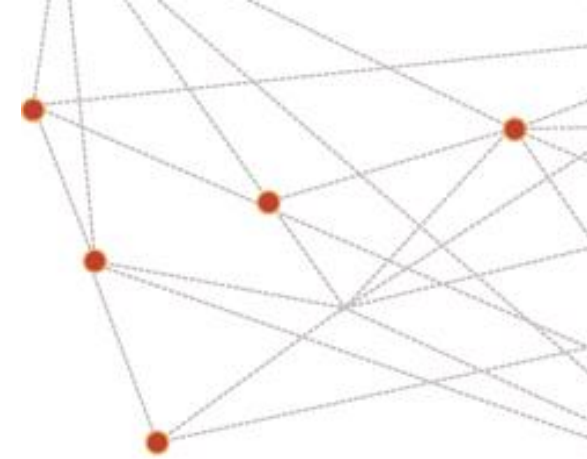
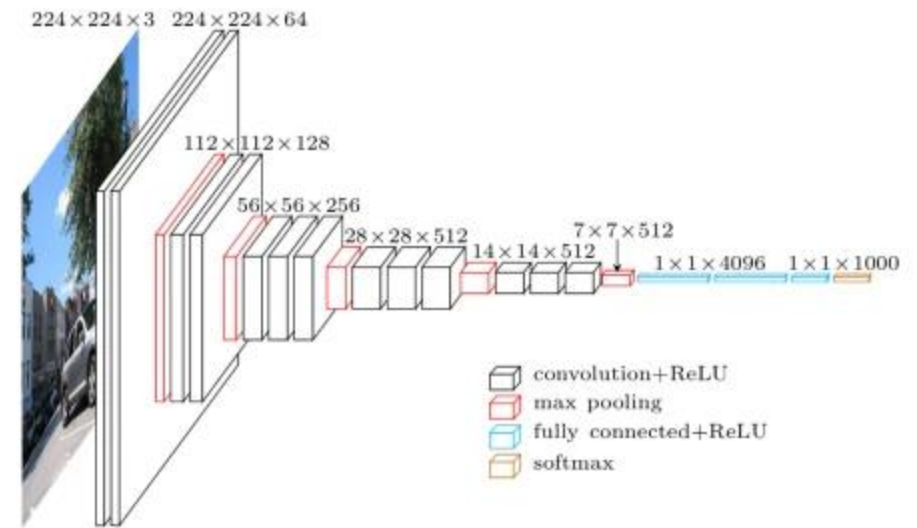




VGG

VGG stands for Visual Geometry Group; it is a standard deep Convolutional Neural Network (CNN) architecture with multiple layers. The “deep” refers to the number of layers with VGG-16 or VGG-19 consisting of 16 and 19 convolutional layers.

The VGG architecture is the basis of ground-breaking object recognition models. Developed as a deep neural network, the VGGNet also surpasses baselines on many tasks and datasets beyond ImageNet. Moreover, it is now still one of the most popular image recognition architectures.

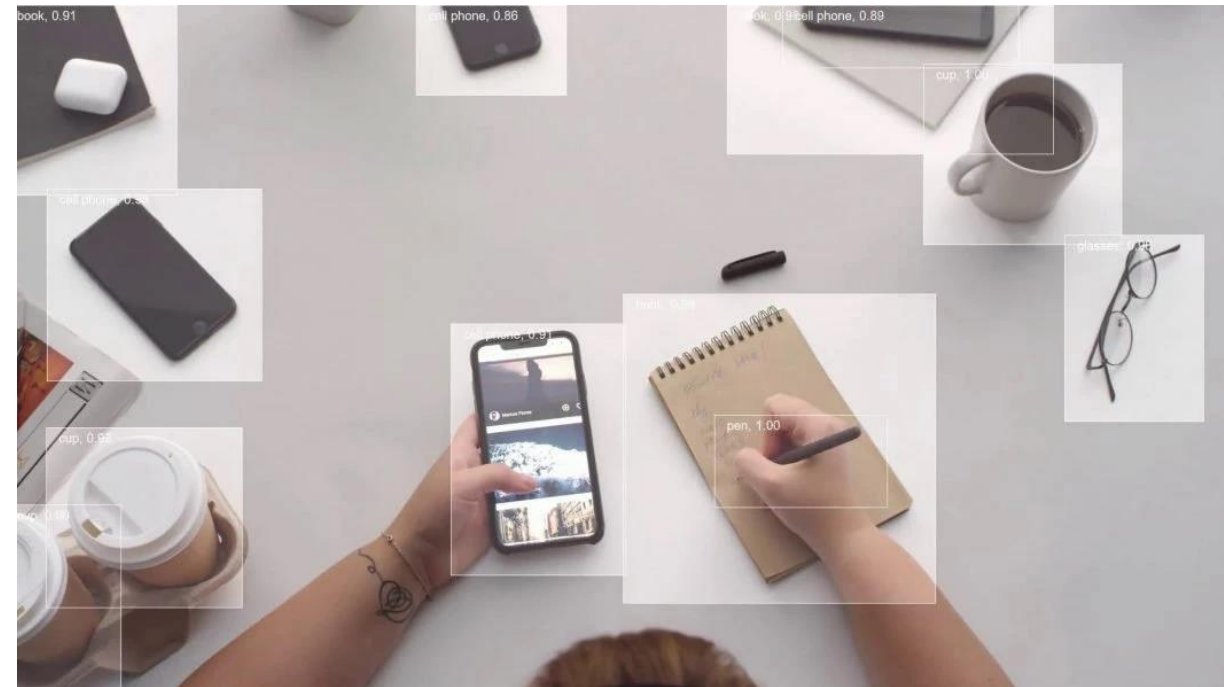
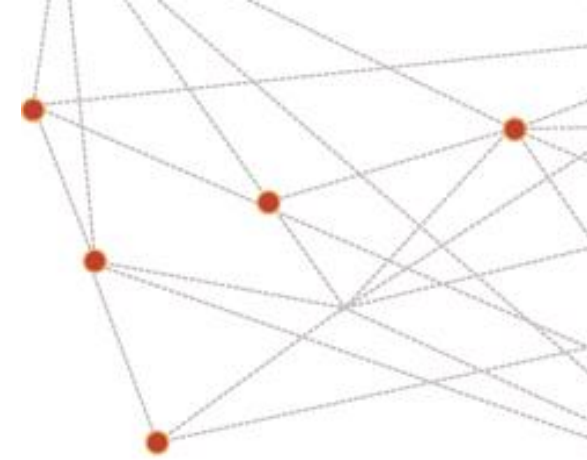




VGG16

The VGG model, or VGGNet, that supports 16 layers is also referred to as VGG16, which is a convolutional neural network model proposed by A. Zisserman and K. Simonyan from the University of Oxford.

The VGG16 model achieves almost 92.7% top-5 test accuracy in ImageNet.

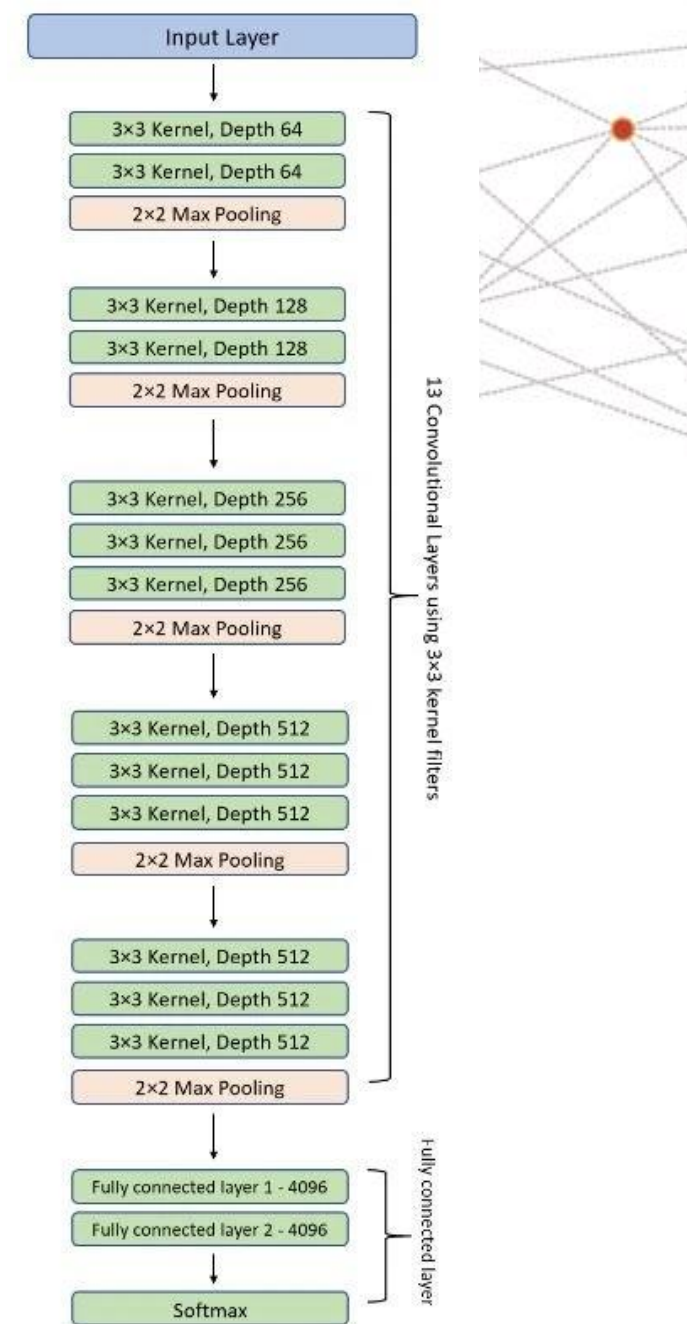




VGG16

The VGG network is constructed with very small convolutional filters. The VGG-16 consists of 13 convolutional layers and three fully connected layers.

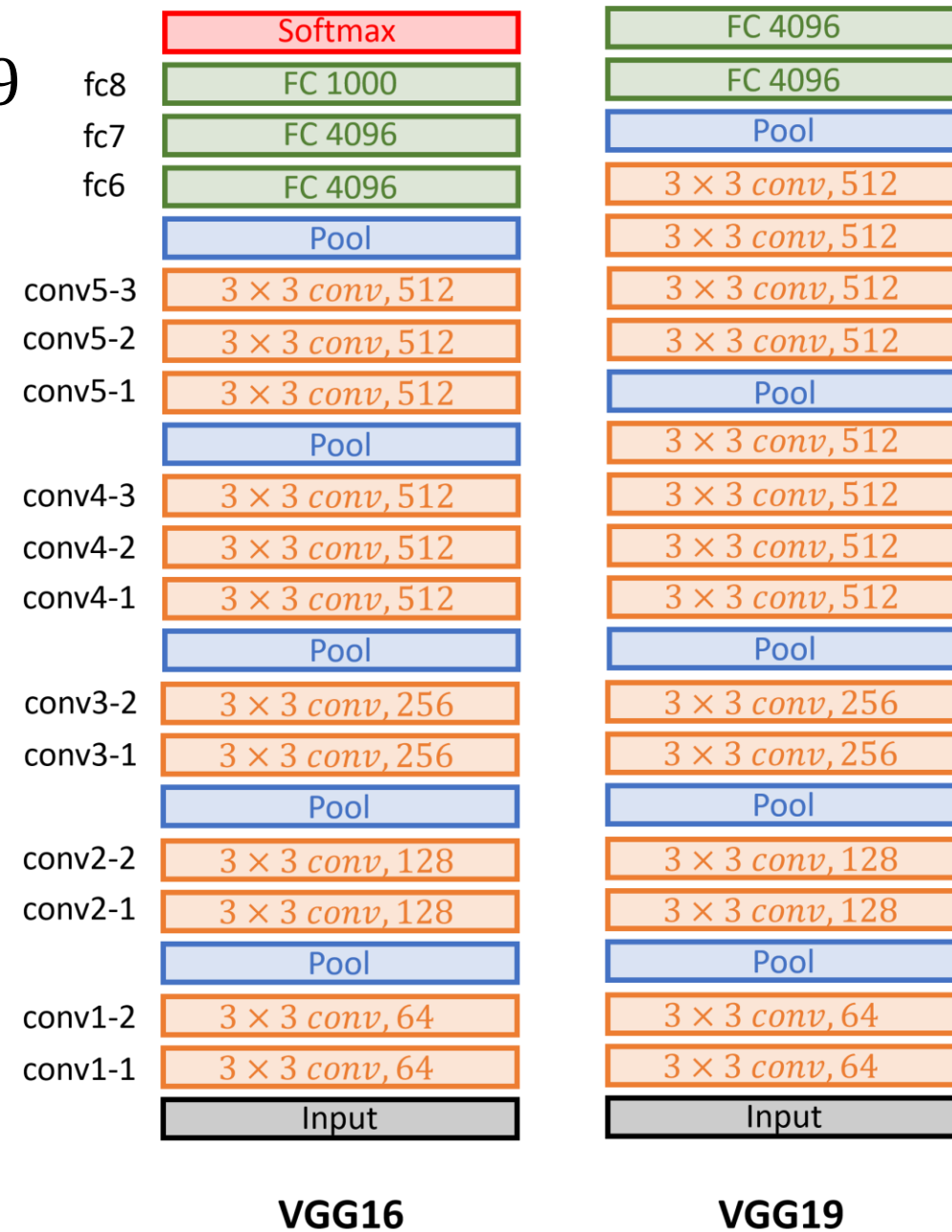
- The VGGNet takes in an image input size of 224×224 .
- VGG's convolutional layers leverage a minimal receptive field, i.e., 3×3 , the smallest possible size that still captures up/down and left/right. Moreover, there are also 1×1 convolution filters acting as a linear transformation of the input. This is followed by a ReLU unit.
- The VGGNet has three fully connected layers. Out of the three layers, the first two have 4096 channels each, and the third has 1000 channels, 1 for each class.





The concept of the VGG19 model (also VGGNet-19) is the same as the VGG16 except that it supports 19 layers. The “16” and “19” stand for the number of weight layers in the model (convolutional layers). This means that VGG19 has three more convolutional layers than VGG16.

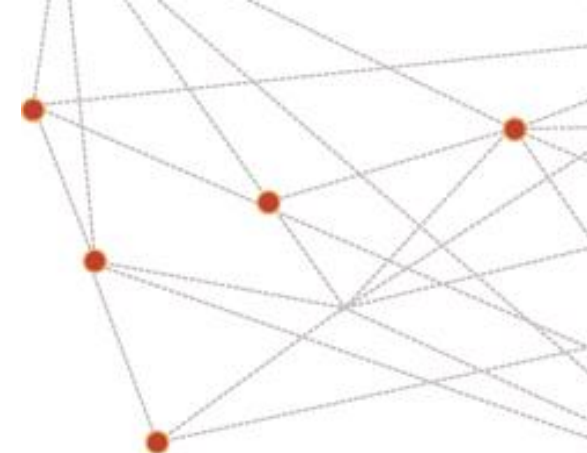
VGG19





VGG remarks

The number of filters that we can use doubles on every step or through every stack of the convolution layer. This is a major principle used to design the architecture of the VGG16 network. One of the crucial downsides of the VGG16 network is that it is a huge network, which means that it takes more time to train its parameters. Because of its depth and number of fully connected layers, the VGG16 model is more than 533MB. This makes implementing a VGG network a time-consuming task.





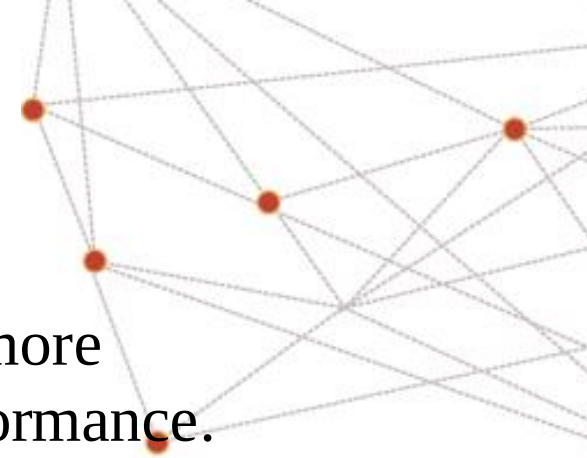
VGGNet problems

As the number of layers increases in CNN, the ability of the model to fit more complex functions also increases. Hence, more layers promise better performance.

Now the question is, why shouldn't you use a VGGNet with more layers, such as VGG20, or VGG50, or VGG100? This is where the problem arises. The weights of a neural network are updated through the backpropagation algorithm, which makes a minor change to each weight so that the loss of the model decreases.

It updates each weight so that it takes a step in the direction along which the loss decreases. This is nothing but the gradient of this weight which can be found using the chain rule.

However, as the gradient keeps flowing backward to the initial layers, the value keeps increasing by each local gradient. This results in the gradient becoming smaller and smaller, thereby making changes to the initial layers very small. This, in turn, increases the training time significantly.



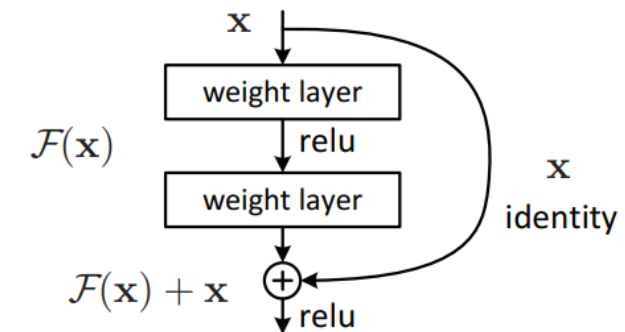
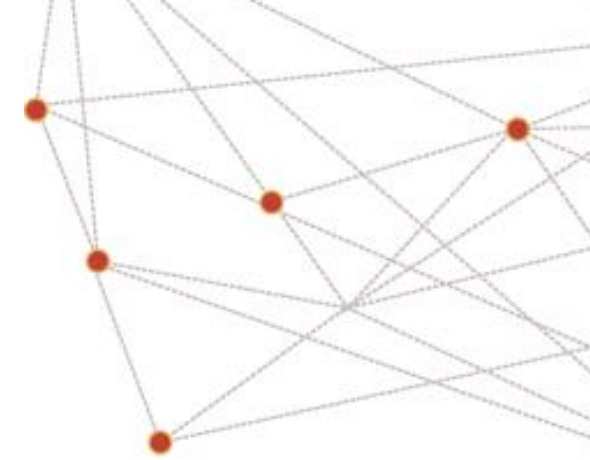


ResNet

In order to solve the problem of the vanishing/exploding gradient, this architecture introduced the concept called Residual Blocks. In this network, we use a technique called skip connections. The skip connection connects activations of a layer to further layers by skipping some layers in between. This forms a residual block. Resnets are made by stacking these residual blocks together.

The approach behind this network is instead of layers learning the underlying mapping, we allow the network to fit the residual mapping.

The advantage of adding this type of skip connection is that if any layer hurt the performance of architecture then it will be skipped by regularization. So, this results in training a very deep neural network without the problems caused by vanishing/exploding gradient.

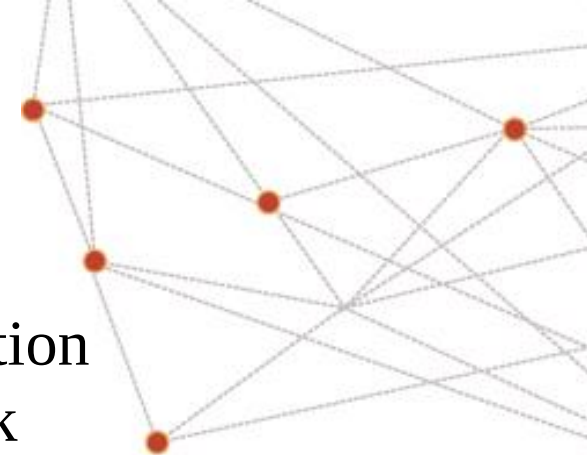




ResNet34

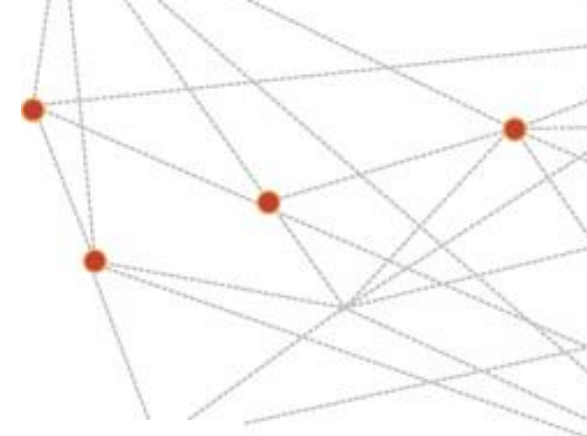
The first ResNet architecture was the Resnet-34, which involved the insertion of shortcut connections in turning a plain network into its residual network counterpart. In this case, the plain network was inspired by VGG neural networks (VGG-16, VGG-19), with the convolutional networks having 3×3 filters. However, compared to VGGNets, ResNets have fewer filters and lower complexity.

The layers had the same number of filters for the same output feature map size, and the number of filters doubled in case the feature map size was halved in order to preserve the time complexity per layer. It consisted of 34 weighted layers.

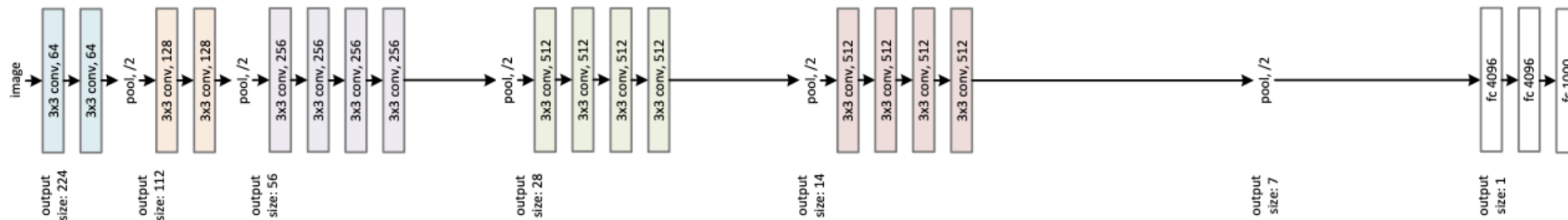




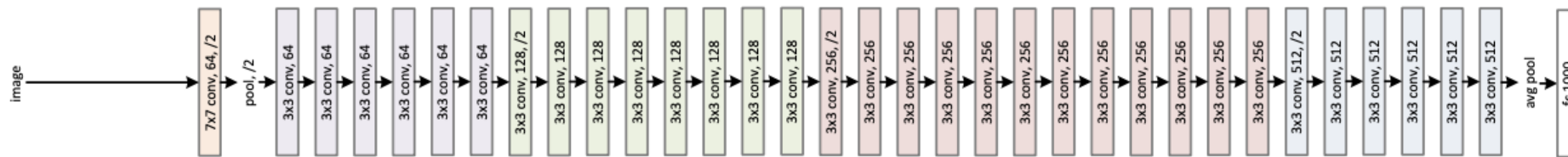
ResNet34



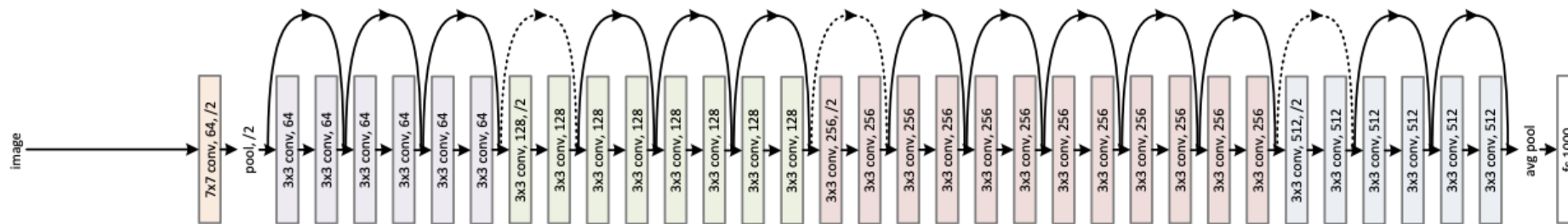
VGG-19



34-layer plain



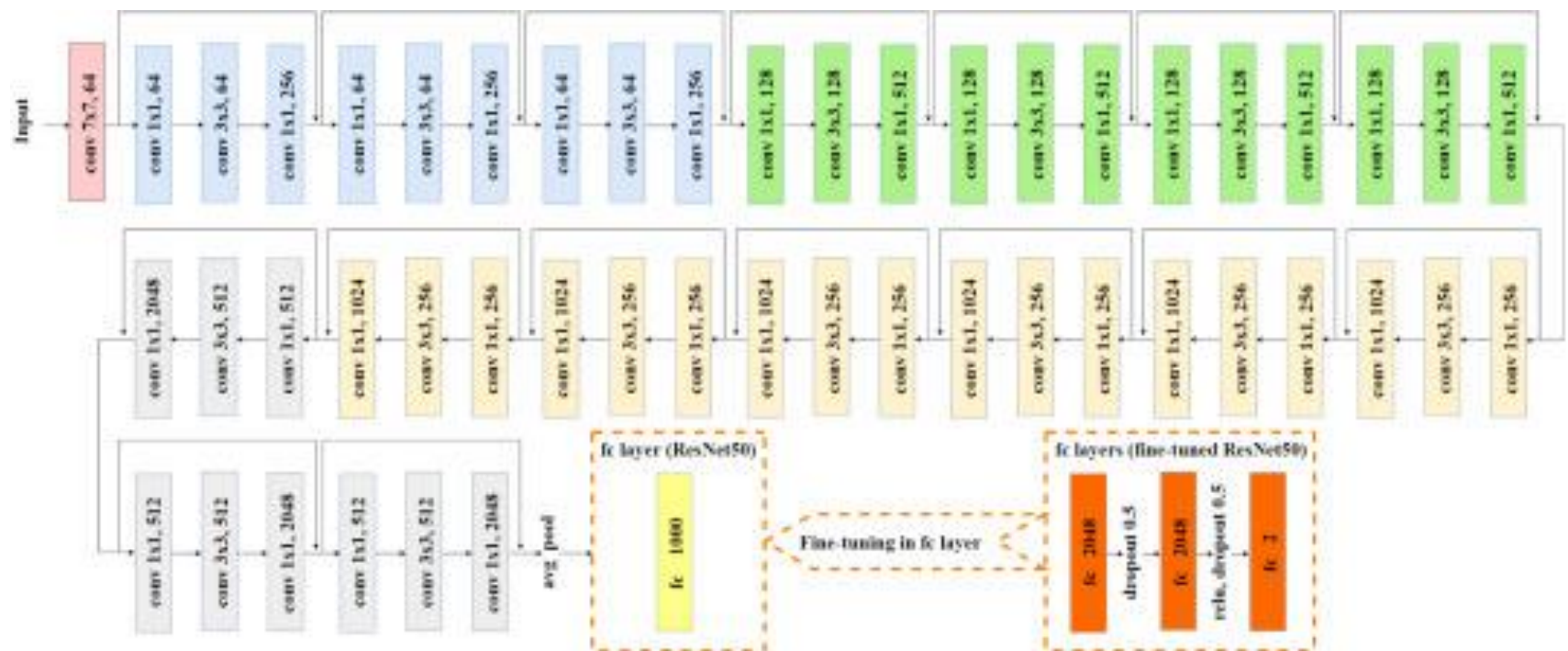
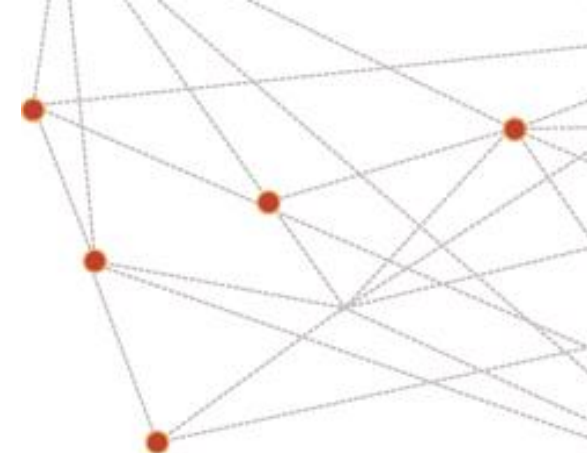
34-layer residual

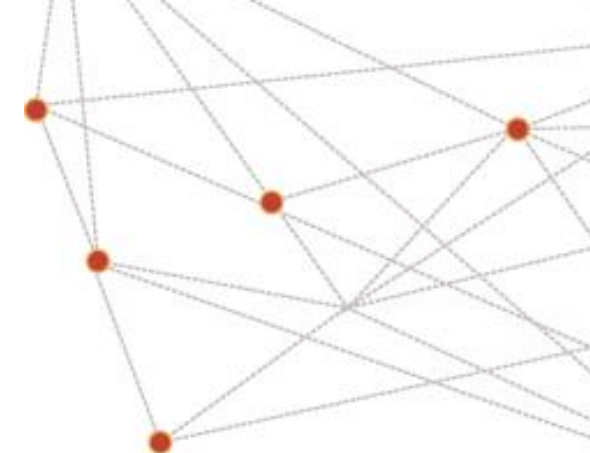




ResNet50

ResNet50 is a variant of ResNet model which has 48 Convolution layers along with 1 MaxPool and 1 Average Pool layer. In this case, the building block was modified into a bottleneck design due to concerns over the time taken to train the layers. This used a stack of 3 layers instead of the earlier 2.





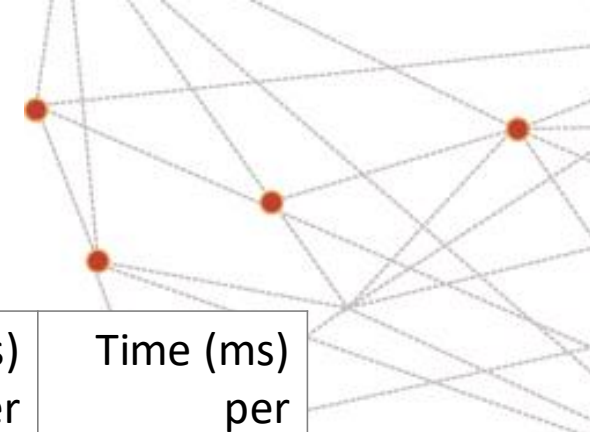
ResNet-101 and ResNet-152

Large Residual Networks such as 101-layer ResNet101 or ResNet152 are constructed by using more 3-layer blocks. And even at increased network depth, the 152-layer ResNet has much lower complexity than VGG-16 or VGG-19 nets.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹	11.3×10 ⁹



ResNets performances



Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6

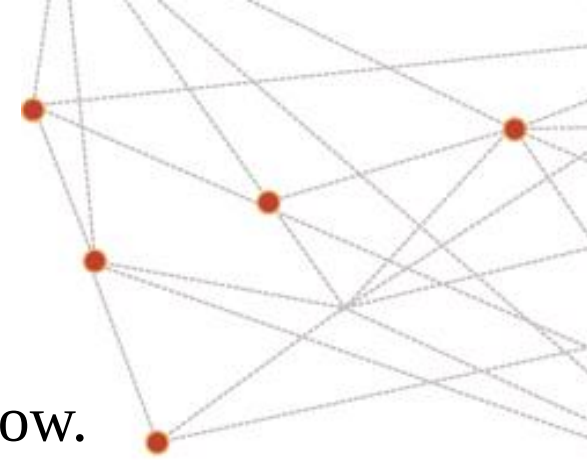


InceptionNets

Salient parts in the image can have extremely large variation in size. For instance, an image with a dog can be either of the following, as shown below. The area occupied by the dog is different in each image.



Because of this huge variation in the location of the information, choosing the right kernel size for the convolution operation becomes tough. A larger kernel is preferred for information that is distributed more globally, and a smaller kernel is preferred for information that is distributed more locally.

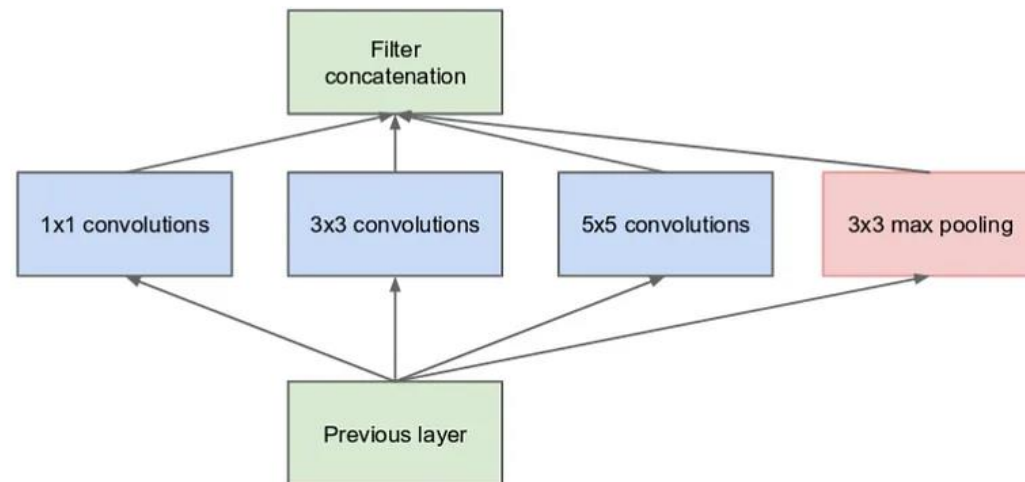




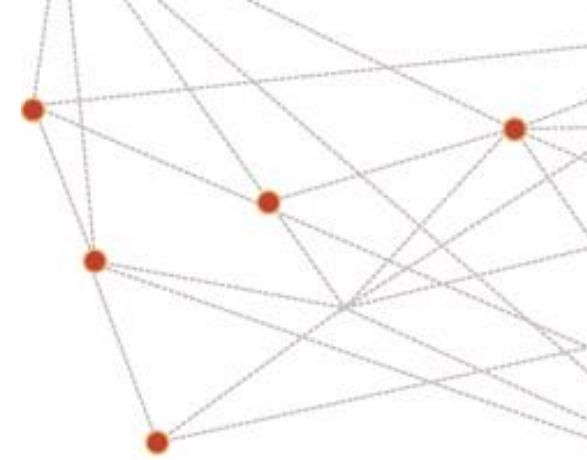
InceptionNets

Very deep networks are prone to overfitting. It also hard to pass gradient updates through the entire network.

Why not have filters with multiple sizes operate on the same level? The network essentially would get a bit “wider” rather than “deeper”. The authors designed the inception module to reflect the same.

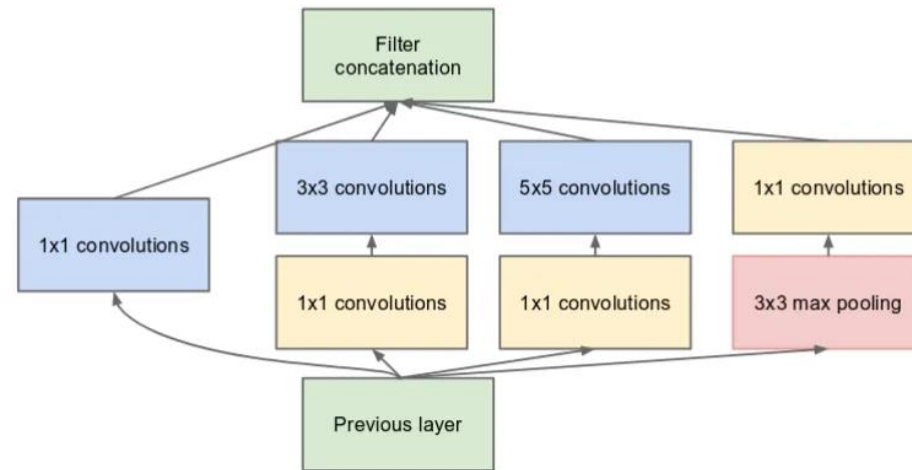


(a) Inception module, naïve version



InceptionNets

To make it cheaper, the authors limit the number of input channels by adding an extra 1x1 convolution before the 3x3 and 5x5 convolutions. Though adding an extra operation may seem counterintuitive, 1x1 convolutions are far more cheaper than 5x5 convolutions, and the reduced number of input channels also help. Do note that however, the 1x1 convolution is introduced after the max pooling layer, rather than before.



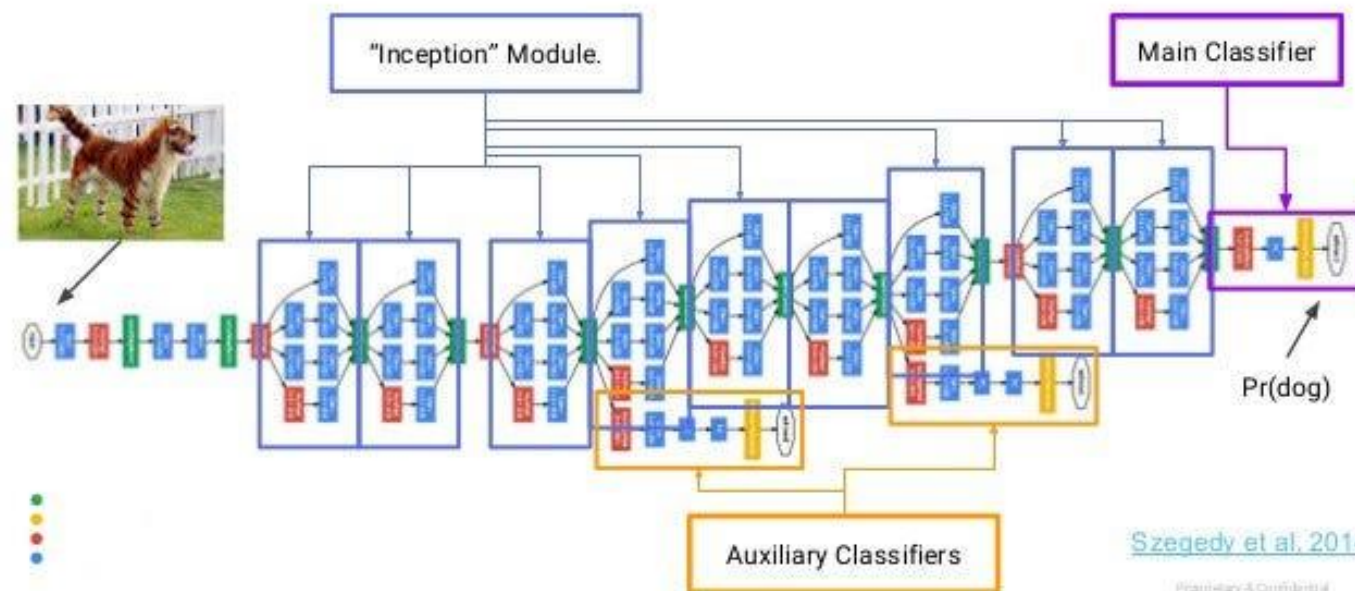
(b) Inception module with dimension reductions

InceptionNet V1

This was popularly known as GoogLeNet (Inception v1).

GoogLeNet has 9 such inception modules stacked linearly. It is 22 layers deep (27, including the pooling layers). It uses global average pooling at the end of the last inception module.

GoogLeNet (aka "Inception") Architecture



Szegedy et al. 2014

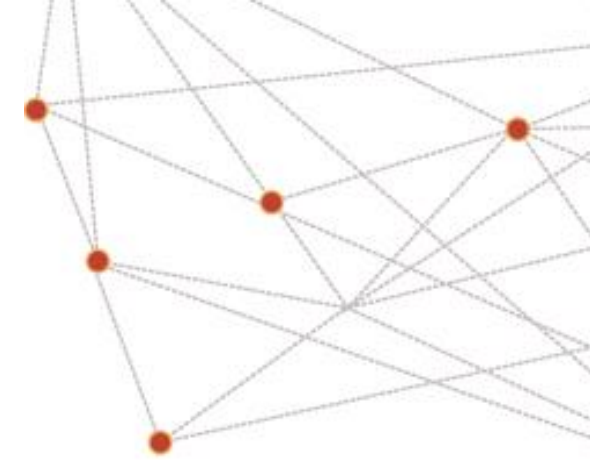
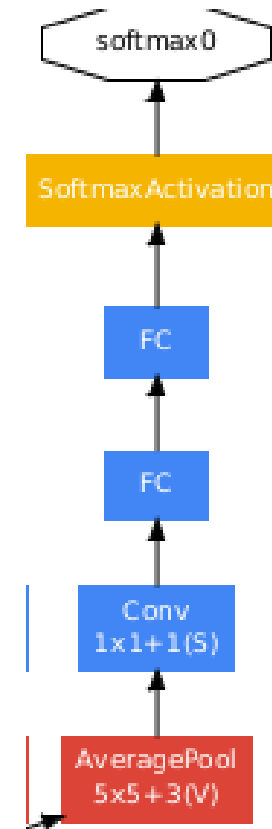
Preliminary & Confidential



InceptionNet V1

As with any very deep network, it is subject to the vanishing gradient problem.

To prevent the middle part of the network from “dying out”, the authors introduced two auxiliary classifiers. They essentially applied softmax to the outputs of two of the inception modules, and computed an auxiliary loss over the same labels. The total loss function is a weighted sum of the auxiliary loss and the real loss. Weight value used was 0.3 for each auxiliary loss.



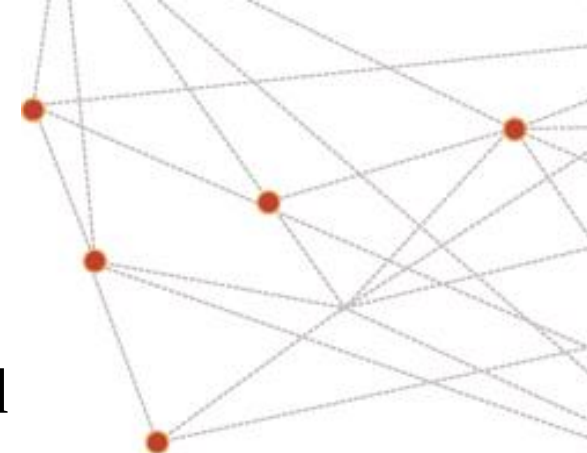


InceptionV2

Inception v2 and Inception v3 have a number of upgrades which increased the accuracy and reduced the computational complexity.

The aims of InceptionV2 are:

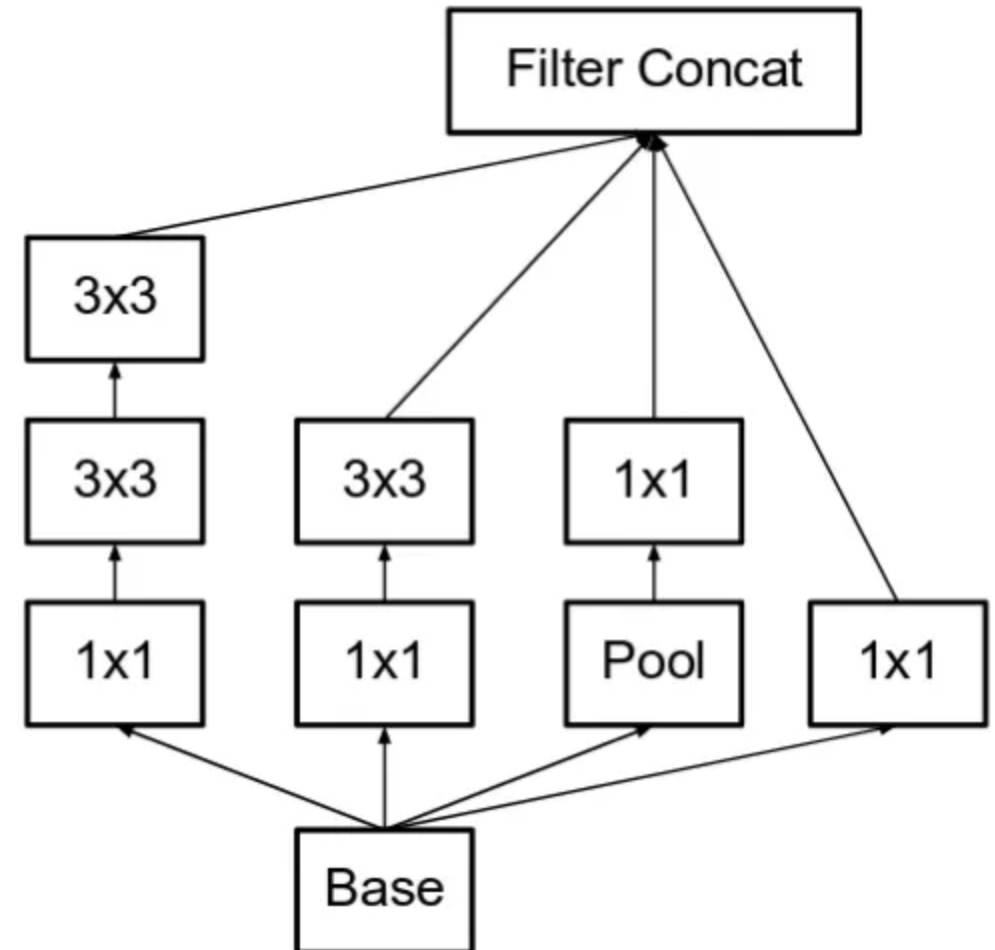
- Reduce representational bottleneck. The intuition was that, neural networks perform better when convolutions didn't alter the dimensions of the input drastically. Reducing the dimensions too much may cause loss of information, known as a “representational bottleneck”
- Using smart factorization methods, convolutions can be made more efficient in terms of computational complexity.



InceptionV2

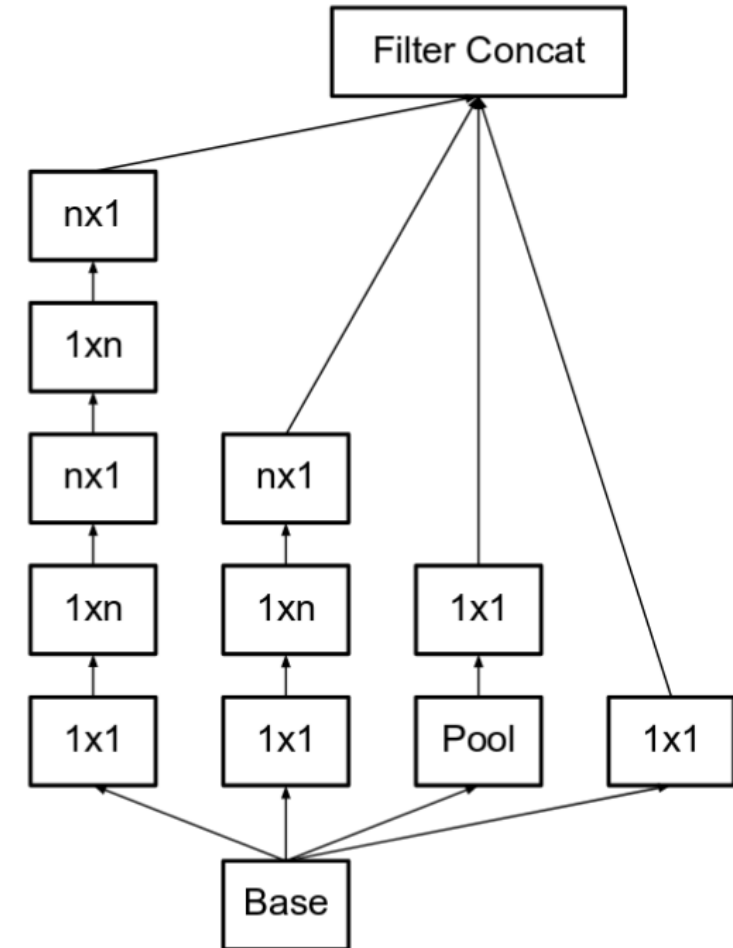
Differently from InceptionV1:

- Factorize 5x5 convolution to two 3x3 convolution operations to improve computational speed. Although this may seem counterintuitive, a 5x5 convolution is 2.78 times more expensive than a 3x3 convolution. So stacking two 3x3 convolutions in fact leads to a boost in performance.



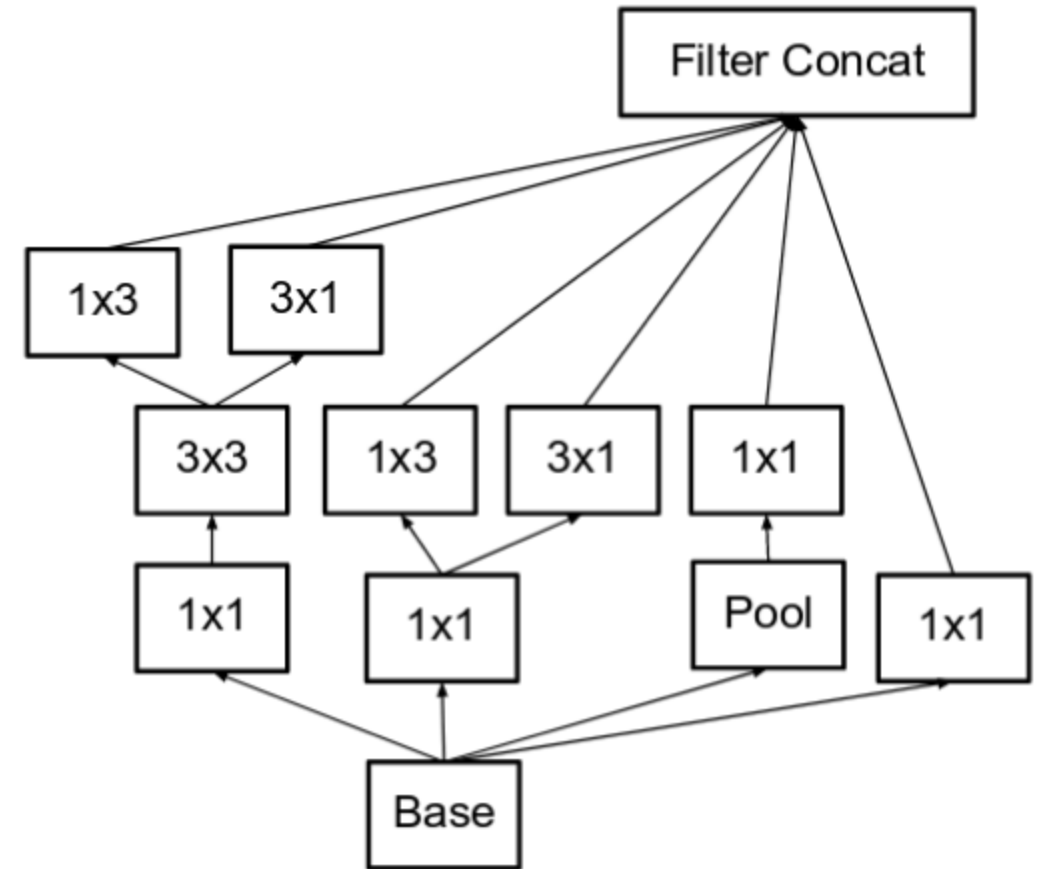
InceptionV2

- Factorize convolutions of filter size $n \times n$ to a combination of $1 \times n$ and $n \times 1$ convolutions. For example, a 3×3 convolution is equivalent to first performing a 1×3 convolution, and then performing a 3×1 convolution on its output. They found this method to be 33% more cheaper than the single 3×3 convolution. This is illustrated in the below image.



InceptionV2

- The filter banks in the module were expanded (made wider instead of deeper) to remove the representational bottleneck. If the module was made deeper instead, there would be excessive reduction in dimensions, and hence loss of information. This is illustrated in the below image.

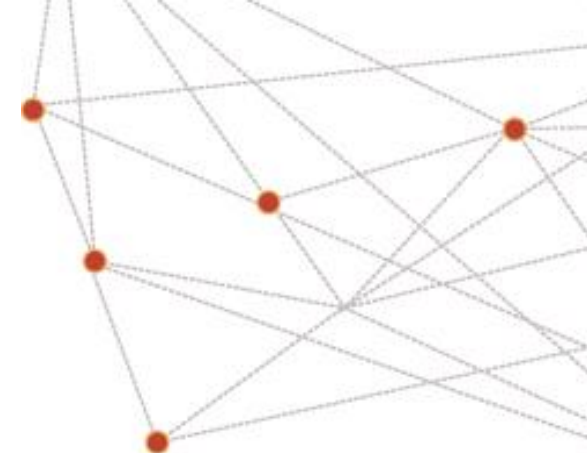




InceptionV3

The premises of InceptionV3 are:

- The auxiliary classifiers didn't contribute much until near the end of the training process, when accuracies were nearing saturation. They function as regularizers, especially if they have BatchNorm or Dropout operations.
- Possibilities to improve on the Inception v2 without drastically changing the modules were to be investigated.

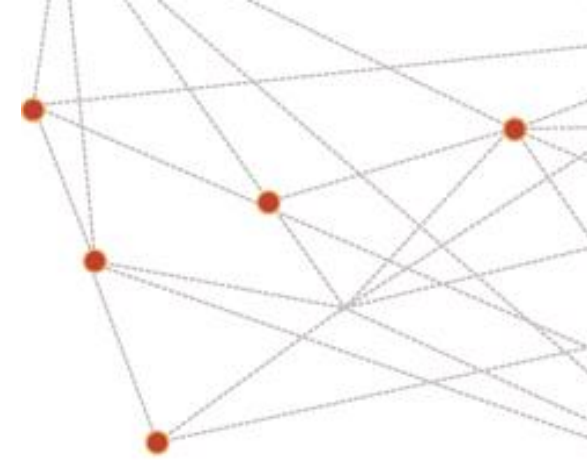




InceptionV3

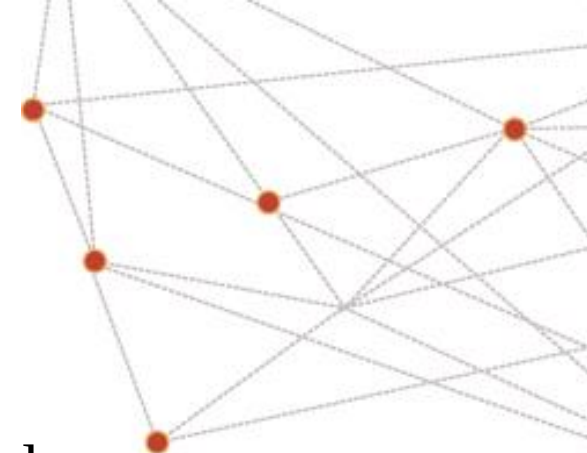
Differently from InceptionV2:

- RMSProp Optimizer.
- Factorized 7x7 convolutions.
- BatchNorm in the Auxillary Classifiers.
- Label Smoothing (A type of regularizing component added to the loss formula that prevents the network from becoming too confident about a class. Prevents over fitting).





InceptionV4



The aim: Make the modules more uniform. The authors also noticed that some of the modules were more complicated than necessary. This can enable us to boost performance by adding more of these uniform modules.

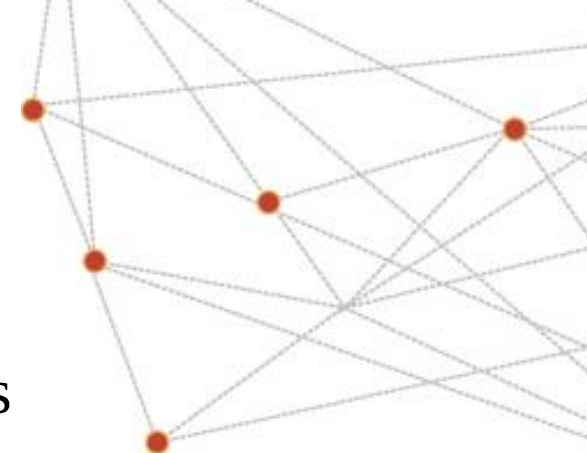
The solution: The “stem” of Inception v4 was modified. The stem here, refers to the initial set of operations performed before introducing the Inception blocks.

They had three main inception modules, named A,B and C (Unlike Inception v2, these modules are infact named A,B and C). They look very similar to their Inception v2 (or v3) counterparts.

Inception v4 introduced specialized “Reduction Blocks” which are used to change the width and height of the grid. The earlier versions didn’t explicitly have reduction blocks, but the functionality was implemented.



InceptionResNet



Inspired by the performance of the ResNet, a hybrid inception module was proposed. There are two sub-versions of Inception ResNet, namely v1 and v2.

The aim: Introduce residual connections that add the output of the convolution operation of the inception module, to the input.

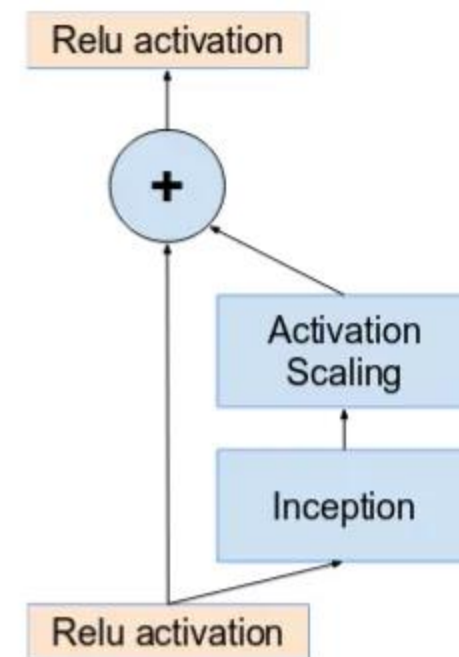
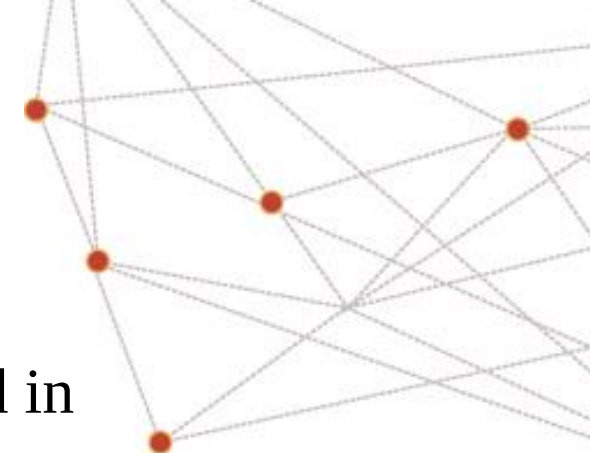
The solution:

- For residual addition to work, the input and output after convolution must have the same dimensions. Hence, we use 1x1 convolutions after the original convolutions, to match the depth sizes (Depth is increased after convolution).



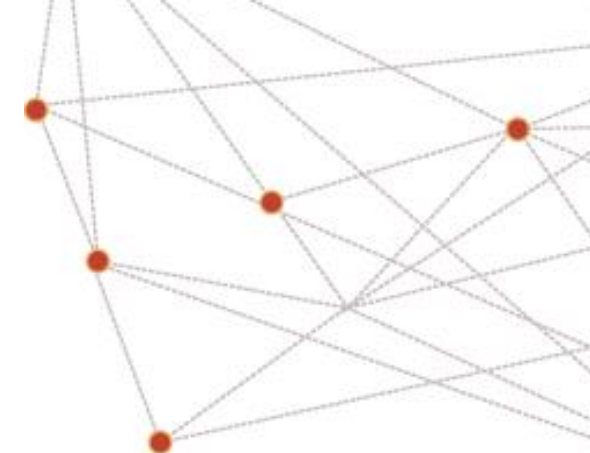
InceptionResNet

- The pooling operation inside the main inception modules were replaced in favour of the residual connections. However, you can still find those operations in the reduction blocks. Reduction block A is same as that of Inception v4.
- Networks with residual units deeper in the architecture caused the network to “die” if the number of filters exceeded 1000. Hence, to increase stability, the authors scaled the residual activations by a value around 0.1 to 0.3.





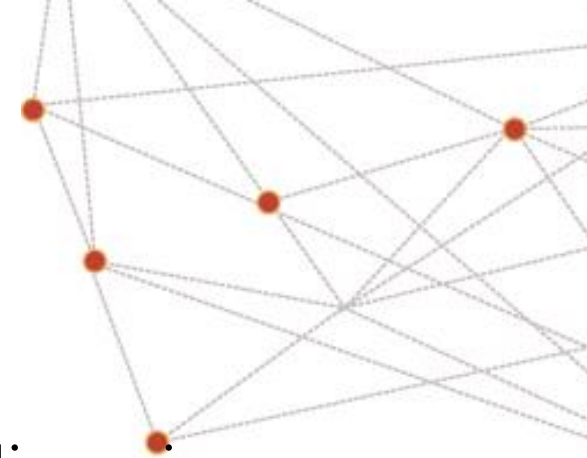
Latest InceptionNet performances



Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Inception V3	92	77.9%	93.7%	23.9M	189	42.2	6.9
Inception ResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0



Light CNNs: MobileNet



MobileNet is based on the concept of Depthwise Separable Convolution.

This convolution originated from the idea that a filter's depth and spatial dimension can be separated, thus, the name separable. Let's take the example of the Sobel filter used in image processing to detect edges.

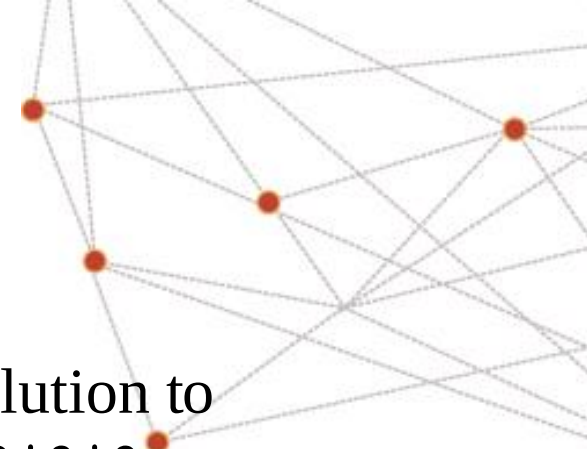
You can separate the height and width dimensions of these filters. Gx filter can be viewed as a matrix product of $\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}^T$ transpose with $\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$.

You'll notice that the filter has disguised itself. It shows it had nine parameters, but it has six. This is possible because of the separation of its height and width dimensions.

The same idea applies to a separate depth dimension from horizontal (width*height), which gives us a depthwise separable convolution where we perform depthwise convolution. After that, we use a 1×1 filter to cover the depth dimension.



Light CNNs: MobileNet



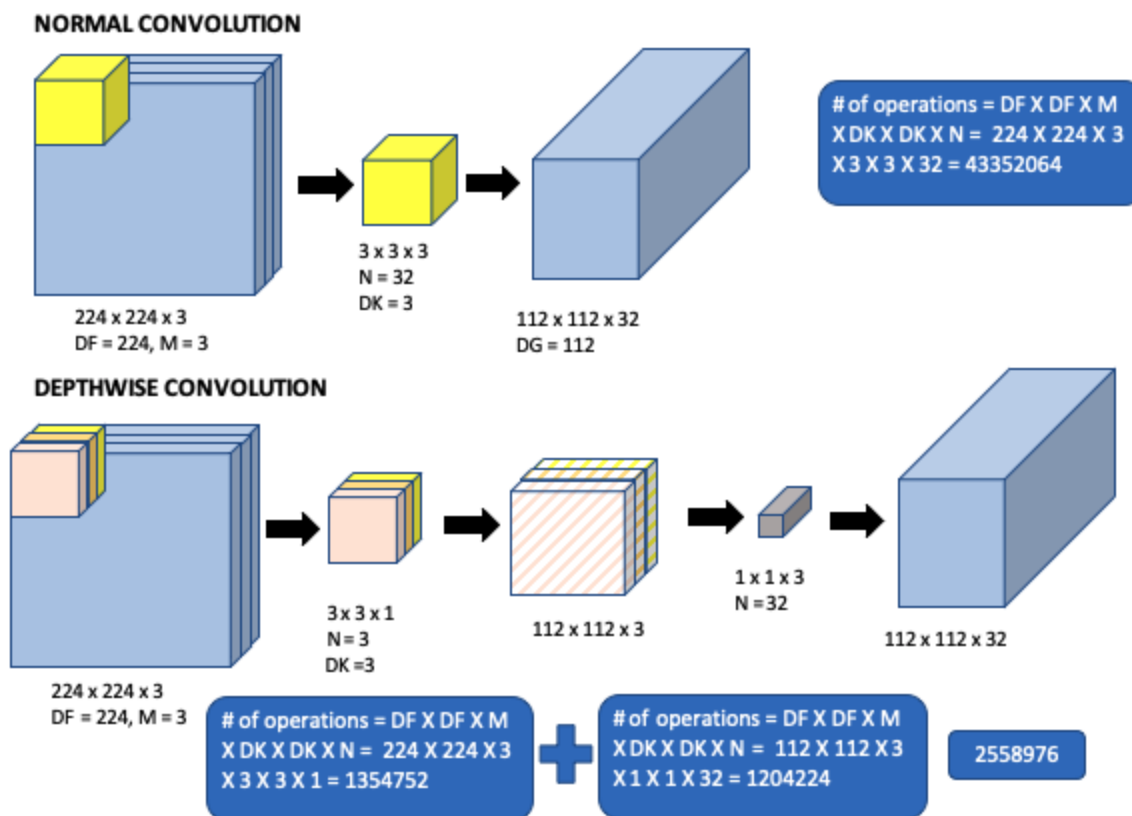
One thing to notice is how much the parameters are reduced by this convolution to output the same number of channels. To produce one channel, we'd need $3*3*3$ parameters to perform depth-wise convolution and $1*3$ parameters to perform further convolution in-depth dimension.

But if we'd need three output channels, we'd only need $3*3$ depth filter, giving us a total of 36 (= 27 +9) parameters, while for the same number of output channels in regular convolution, we'd need $3*3*3$ filters giving us a total of 81 parameters.

Depthwise is a map of a single convolution on each input channel separately. Therefore, its number of output channels is the same as the number of the input channels. Its computational cost is: $Df^2 * M * Dk^2$.

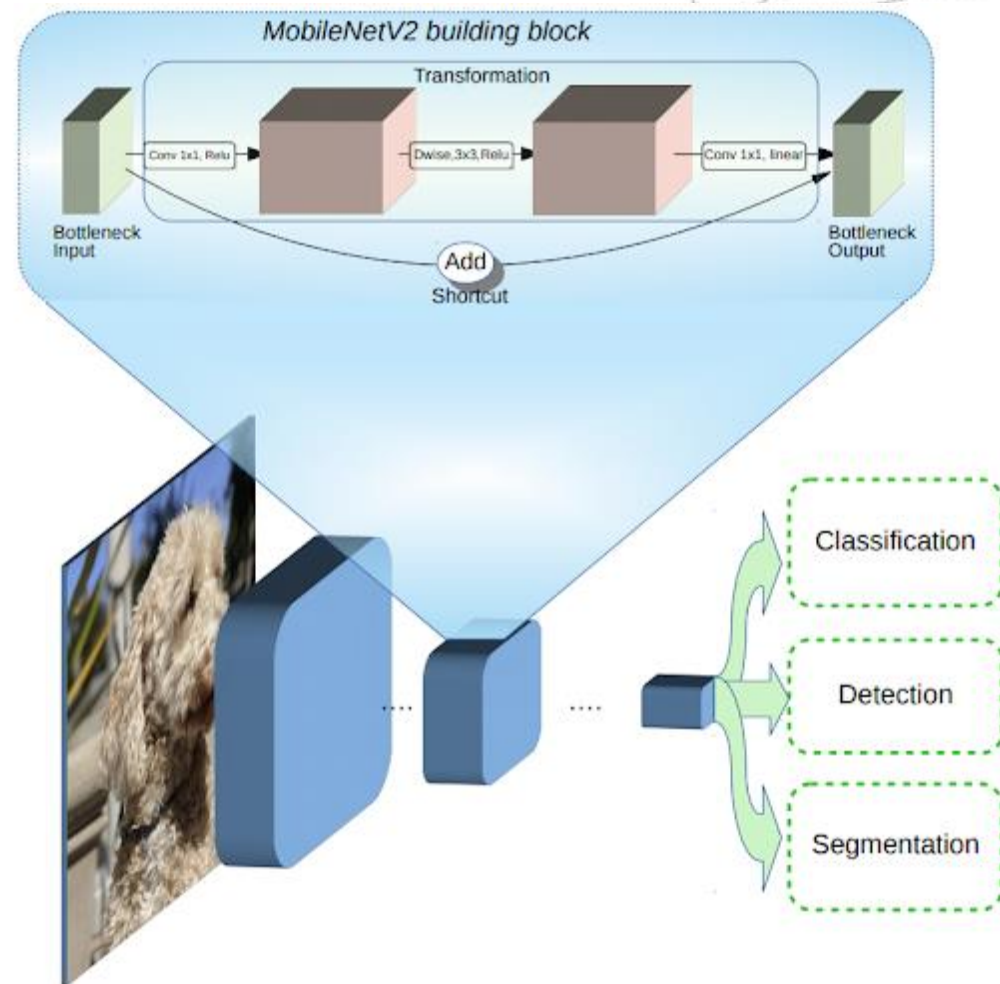
Light CNNs: MobileNet

The main difference between MobileNet architecture and a traditional CNN versus a single 3x3 convolution layer followed by the batch norm and a rectified linear unit (ReLU) is that MobileNet splits the convolution into a 3x3 depthwise convolution and a 1x1 pointwise convolution.



MobileNetV2

MobileNetV2 builds upon the ideas from MobileNetV1, using depthwise separable convolution as efficient building blocks. However, V2 introduces two new features to the architecture: 1) linear bottlenecks between the layers, and 2) shortcut connections between the bottlenecks. The intuition is that the bottlenecks encode the model's intermediate inputs and outputs while the inner layer encapsulates the model's ability to transform from lower-level concepts such as pixels to higher level descriptors such as image categories. Finally, as with traditional residual connections, shortcuts enable faster training and better accuracy.

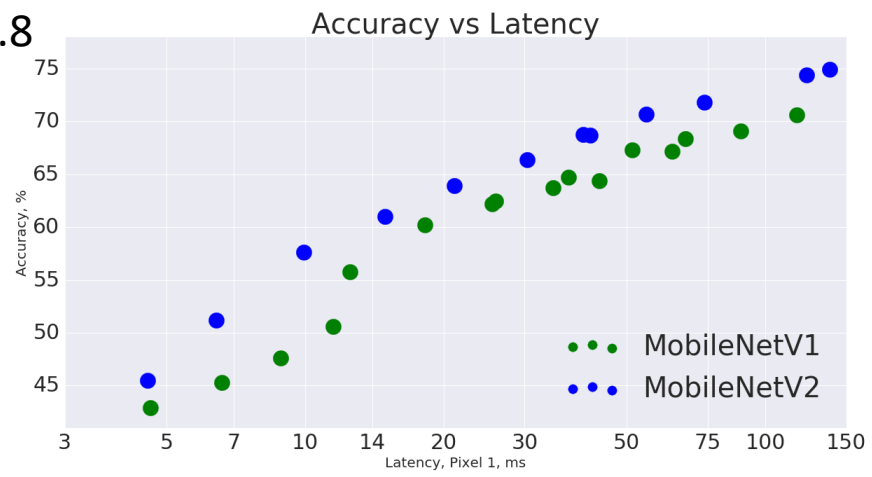




MobileNetV1 vs MobileNetV2

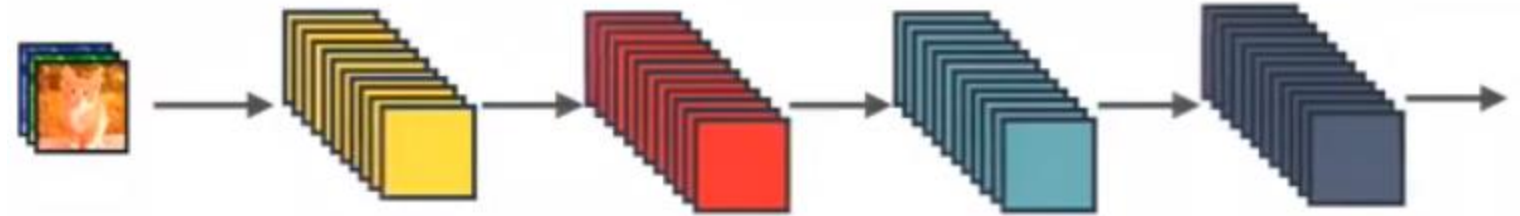
Overall, the MobileNetV2 models are faster for the same accuracy across the entire latency spectrum. In particular, the new models use 2x fewer operations, need 30% fewer parameters and are about 30-40% faster on a Google Pixel phone than MobileNetV1 models, all while achieving higher accuracy.

Model	Size	Top-1 Acc	Top-5 Acc	Param	Depth	Time (CPU)	Time (GPU)
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8

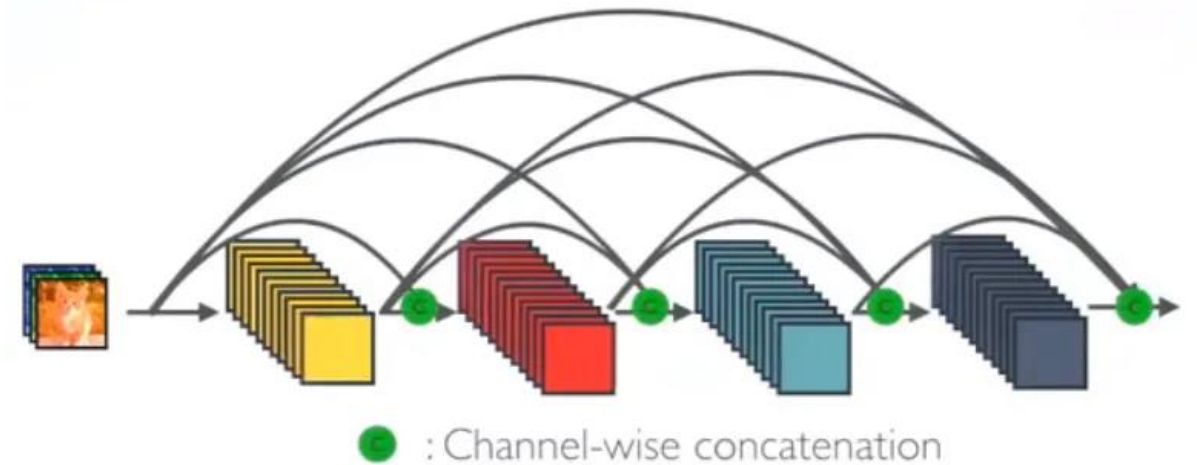


DenseNets

In Standard ConvNet, input image goes through multiple convolution and obtain high-level features.



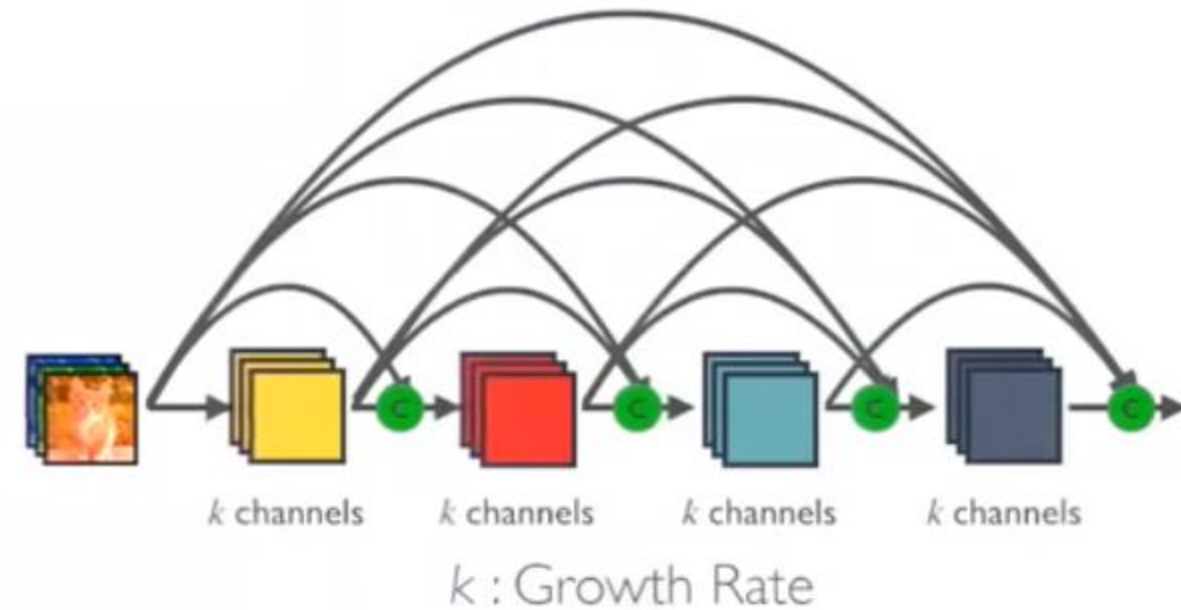
In ResNet, identity mapping is proposed to promote the gradient propagation. Element-wise addition is used. It can be viewed as algorithms with a state passed from one ResNet module to another one.



DenseNets

In DenseNet, each layer obtains additional inputs from all preceding layers and passes on its own feature-maps to all subsequent layers. Concatenation is used. Each layer is receiving a “collective knowledge” from all preceding layers.

Since each layer receives feature maps from all preceding layers, network can be thinner and compact, i.e. number of channels can be fewer. The growth rate k is the additional number of channels for each layer. So, it have higher computational efficiency and memory efficiency.



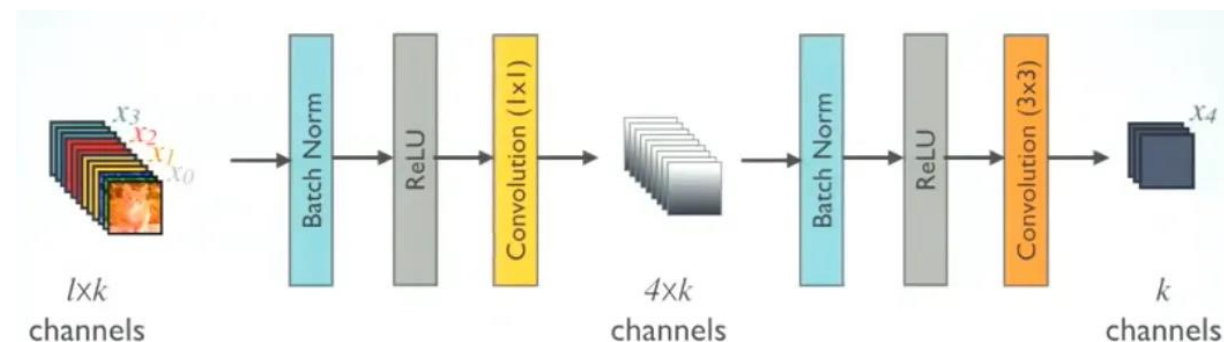
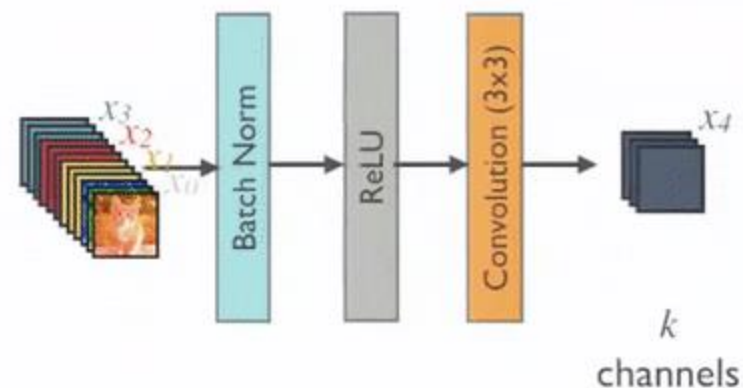
DenseNets

DenseNet is composed by:

- Basic composition layers:

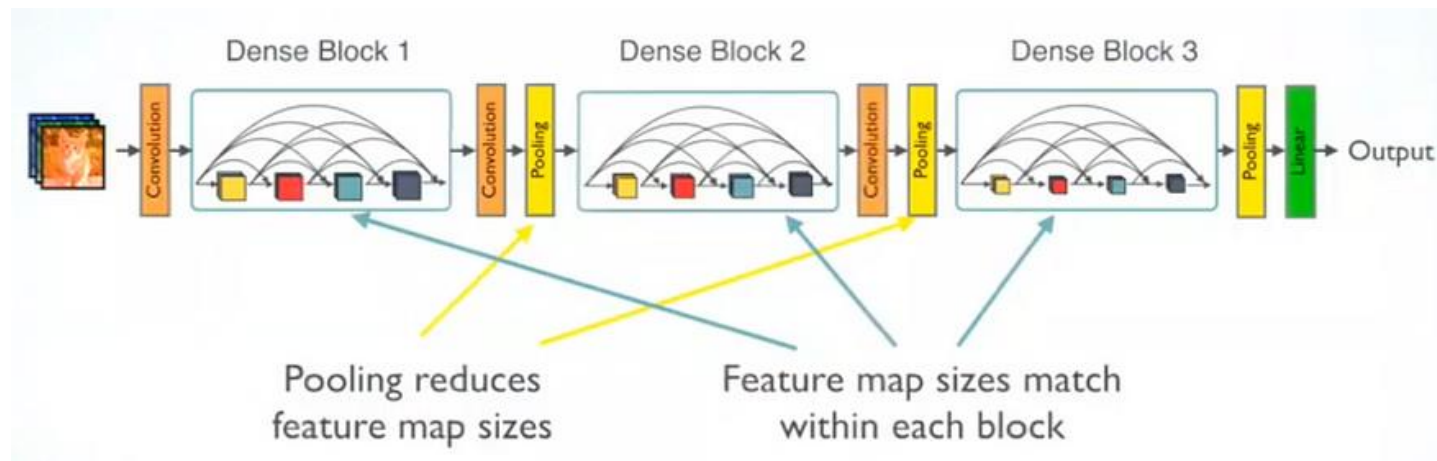
For each composition layer, Pre-Activation Batch Norm (BN) and ReLU, then 3×3 Conv are done with output feature maps of k channels, say for example, to transform x_0 , x_1 , x_2 , x_3 to x_4 . This is the idea from Pre-Activation ResNet.

- Bottleneck Layers: To reduce the model complexity and size, BN-ReLU- 1×1 Conv is done before BN-ReLU- 3×3 Conv.



DenseNets

- Multiple Dense Blocks with Transition Layers: 1×1 Conv followed by 2×2 average pooling are used as the transition layers between two contiguous dense blocks. Feature map sizes are the same within the dense block so that they can be concatenated together easily. At the end of the last dense block, a global average pooling is performed and then a softmax classifier is attached.

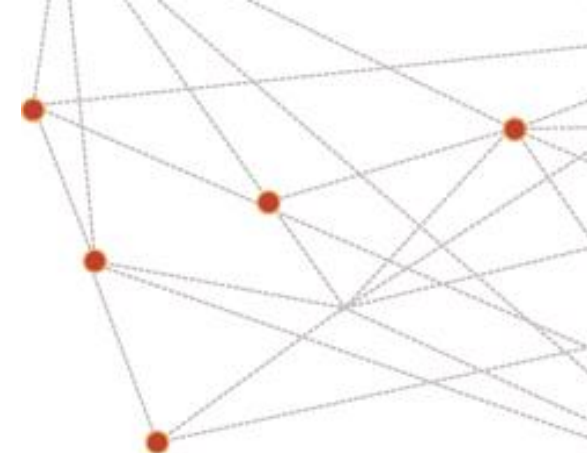




DenseNets

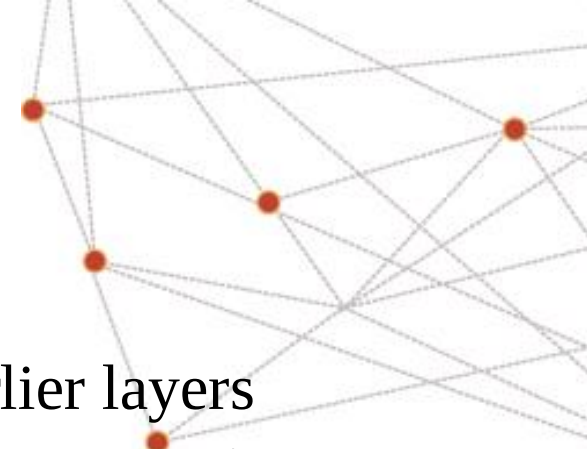
- DenseNet-BC (Further Compression): If a dense block contains m feature-maps, The transition layer generate θm output feature maps, where $0 < \theta \leq 1$ is referred to as the compression factor. When $\theta = 1$, the number of feature-maps across transition layers remains unchanged. DenseNet with $\theta < 1$ is referred as DenseNet-C, and $\theta = 0.5$ in the experiment. When both the bottleneck and transition layers with $\theta < 1$ are used, the model is referred as DenseNet-BC.

Finally, DenseNets with/without B/C and with different L layers and k growth rate are trained.





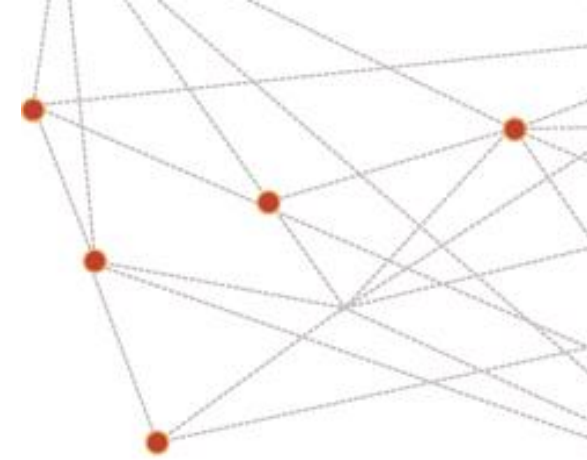
DenseNets: advantages



- **Strong Gradient Flow:** The error signal can be easily propagated to earlier layers more directly. This is a kind of implicit deep supervision as earlier layers can get direct supervision from the final classification layer.
- **Parameter & Computational Efficiency:** For each layer, number of parameters in ResNet is directly proportional to $C \times C$ while Number of parameters in DenseNet is directly proportional to $l \times k \times k$. Since $k \ll C$, DenseNet has much smaller size than ResNet.
- **More Diversified Features:** Since each layer in DenseNet receive all preceding layers as input, more diversified features and tends to have richer patterns.
- **Maintains Low Complexity Features:** In Standard ConvNet, classifier uses most complex features. In DenseNet, classifier uses features of all complexity levels. It tends to give more smooth decision boundaries. It also explains why DenseNet performs well when training data is insufficient.



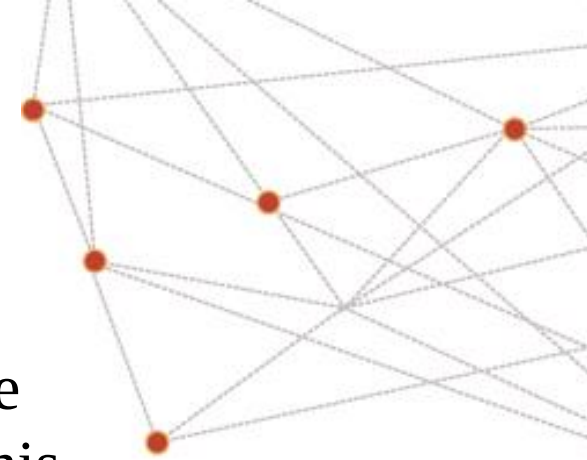
DenseNets models and performances



Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3
DenseNet201	80	77.3%	93.6%	20.2M	402	127.2	6.7



EfficientNets



Some models are scaled depth-wise, and some are scaled widthwise. Some models simply take in images of a larger resolution to get better results. This technique of randomly scaling models requires manual tuning and many person-hours, often resulting in little or no improvement in performance.

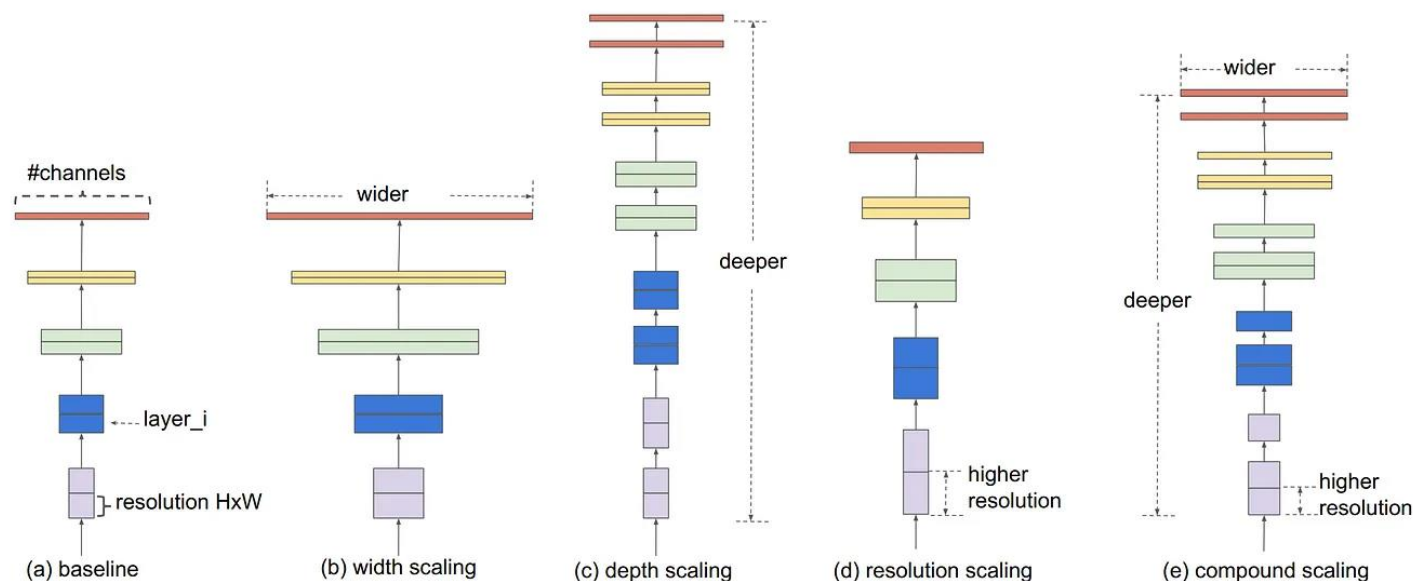
EfficientNet uses a technique called compound coefficient to scale up models in a simple but effective manner. Instead of randomly scaling up width, depth or resolution, compound scaling uniformly scales each dimension with a certain fixed set of scaling coefficients. While scaling single dimensions helps improve model performance, balancing the scale in all the three dimensions — width, depth, and image resolution — considering the variable available resources best improve the overall model performance.

EfficientNets

The compound scaling method is based on the idea of balancing dimensions of width, depth, and resolution by scaling with a constant ratio.

The intuition for the networks is, if the input image is bigger, then the network needs more layers to increase the receptive field and more channels to capture more fine-grained patterns on the bigger image.

The compound scaling technique also helped improve the model efficiency and accuracy of previous CNN models such as MobileNet and ResNet by around 1.4% and 0.7% ImageNet accuracy, respectively, compared to other random scaling methods.

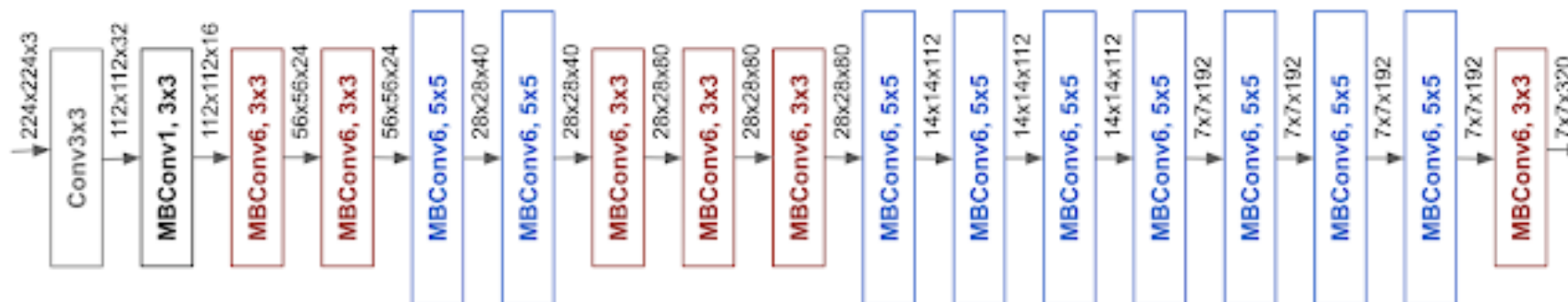




EfficientNets

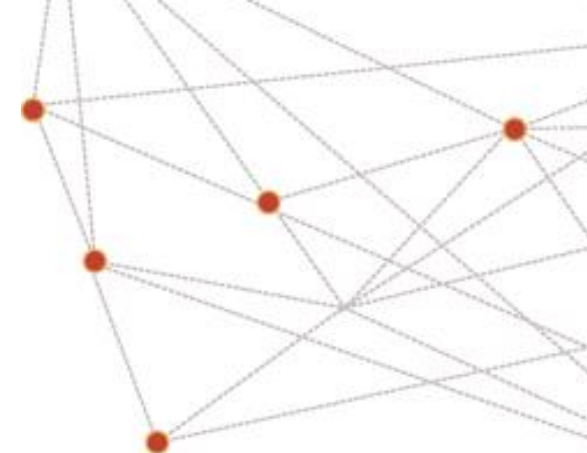
The EfficientNet-B0 architecture wasn't developed by engineers but by the neural network itself. They developed this model using a multi-objective neural architecture search that optimizes both accuracy and floating-point operations.

Taking B0 as a baseline model, the authors developed a full family of EfficientNets from B1 to B7 which achieved state of the art accuracy on ImageNet while being very efficient to its competitors.





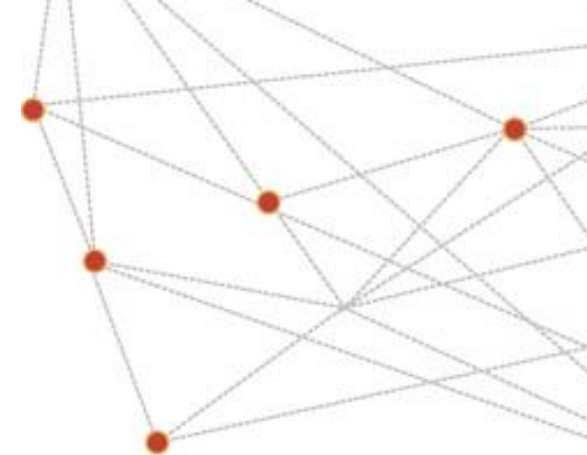
EfficientNets



Model	Top-1 Acc.	Top-5 Acc.	#Params	Ratio-to-EfficientNet	#FLOPS	Ratio-to-EfficientNet
EfficientNet-B0	77.3%	93.5%	5.3M	1x	0.39B	1x
ResNet-50 (He et al., 2016)	76.0%	93.0%	26M	4.9x	4.1B	11x
DenseNet-169 (Huang et al., 2017)	76.2%	93.2%	14M	2.6x	3.5B	8.9x
EfficientNet-B1	79.2%	94.5%	7.8M	1x	0.70B	1x
ResNet-152 (He et al., 2016)	77.8%	93.8%	60M	7.6x	11B	16x
DenseNet-264 (Huang et al., 2017)	77.9%	93.9%	34M	4.3x	6.0B	8.6x
Inception-v3 (Szegedy et al., 2016)	78.8%	94.4%	24M	3.0x	5.7B	8.1x
Xception (Chollet, 2017)	79.0%	94.5%	23M	3.0x	8.4B	12x
EfficientNet-B2	80.3%	95.0%	9.2M	1x	1.0B	1x
Inception-v4 (Szegedy et al., 2017)	80.0%	95.0%	48M	5.2x	13B	13x
Inception-resnet-v2 (Szegedy et al., 2017)	80.1%	95.1%	56M	6.1x	13B	13x
EfficientNet-B3	81.7%	95.6%	12M	1x	1.8B	1x
ResNeXt-101 (Xie et al., 2017)	80.9%	95.6%	84M	7.0x	32B	18x
PolyNet (Zhang et al., 2017)	81.3%	95.8%	92M	7.7x	35B	19x
EfficientNet-B4	83.0%	96.3%	19M	1x	4.2B	1x
SENet (Hu et al., 2018)	82.7%	96.2%	146M	7.7x	42B	10x
NASNet-A (Zoph et al., 2018)	82.7%	96.2%	89M	4.7x	24B	5.7x
AmoebaNet-A (Real et al., 2019)	82.8%	96.1%	87M	4.6x	23B	5.5x
PNASNet (Liu et al., 2018)	82.9%	96.2%	86M	4.5x	23B	6.0x
EfficientNet-B5	83.7%	96.7%	30M	1x	9.9B	1x
AmoebaNet-C (Cubuk et al., 2019)	83.5%	96.5%	155M	5.2x	41B	4.1x
EfficientNet-B6	84.2%	96.8%	43M	1x	19B	1x
EfficientNet-B7	84.4%	97.1%	66M	1x	37B	1x
GPipe (Huang et al., 2018)	84.3%	97.0%	557M	8.4x	-	-



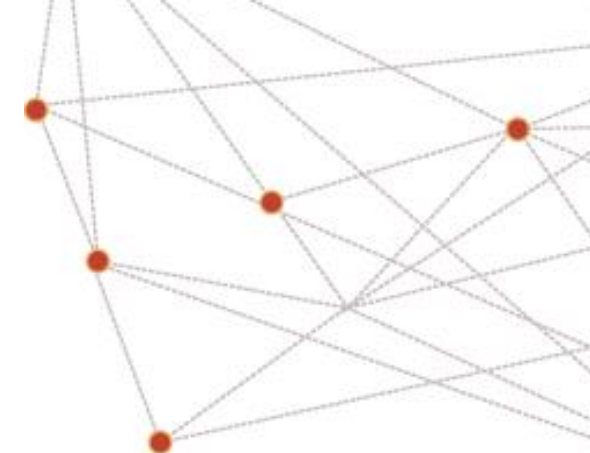
Best results in literature



RANK	METHOD	TOP 1 ACCURACY	TOP 5 ACCURACY	NUMBER OF PARAMS	EXTRA TRAINING DATA	PAPER TITLE	YEAR	PAPER	CODE
1	FixEfficientNet-L2	88.5%	98.7%	480M	✓	Fixing the train-test resolution discrepancy: FixEfficientNet	2020		
2	NoisyStudent (EfficientNet-L2)	88.4%	98.7%	480M	✓	Self-training with Noisy Student improves ImageNet classification	2020		
3	BiT-L (ResNet)	87.54%	98.46%	928M	✓	Big Transfer (BiT): General Visual Representation Learning	2019		
4	NoisyStudent (EfficientNet-L2)	87.4%	98.2%	480M	✓	Self-training with Noisy Student improves ImageNet classification	2019		
5	FixEfficientNet-B7	87.1%	98.2%	66M	✓	Fixing the train-test resolution discrepancy: FixEfficientNet	2020		
6	NoisyStudent (EfficientNet-B7)	86.9%	98.1%	66M	✓	Self-training with Noisy Student improves ImageNet classification	2020		
7	FixEfficientNet-B6	86.7%	98.0%	43M	✓	Fixing the train-test resolution discrepancy: FixEfficientNet	2020		
8	FixResNeXt-101 32x48d	86.4%	98.0%	829M	✓	Fixing the train-test resolution discrepancy	2019		



Best results in literature



Model	Top 1 Accuracy	Number of params	GFLOPs	energy consumption	Extra Training Data	Paper	Code	Result	Year	Tags
OmniVec (ViT)	92.4%				×	OmniVec: Learning robust representations with cross modal sharing			2023	
BASIC-L (Lion, fine-tuned)	91.1%	2440M			×					Conv+Transformer ALIGN JFT-3B
CoCa (finetuned)	91.0%	2100M			×	CoCa: Contrastive Captioners are Image-Text Foundation Models			2022	ALIGN Transformer JFT-3B
Model soups (BASIC-L)	90.98%	2440M			×	Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time			2022	ALIGN JFT-3B Conv+Transformer
Model soups (ViT-G/14)	90.94%	1843M			×	Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time			2022	JFT-3B Transformer